

PROPOSAL FOR ADVANCED ROBOTICS

춘식이 화성가즈아

(협동-3) 디지털 트윈 기반 로봇 자동화 시뮬레이션 시스템 구현

분산형 VLA 기반 로봇 에이전트 시스템

Florence 2

DeepSeek-R1

Qwen-Coder

(멘토) 이체은, 김동우

(춘식아 화성가자) 홍성욱, 양준호, 안교진, 최찬우, 송종진



목차

CONTENTS

- 01 프로젝트 개요
- 02 프로젝트 팀 구성 및 역할
- 03 프로젝트 수행 절차 및 방법
- 04 프로젝트 수행 경과
- 05 자체 평가 의견

CHAPTER 1.1

프로젝트의 개요

분산 처리로 자원 효율화 된 **Local AI 파트너**의 탄생

I 기존 로봇 제어 및 단일 PC 한계와 VLA 도입

고정된 시나리오의 한계

사전 정의된 명령 체계의 경직성으로 인한 환경 대응력 부재.
예기치 못한 환경 변화 발생 시 실시간 대응 불가.

자연어 기반 자율성 요구

인간의 추상적 의도를 스스로 해석하는 지능형 에이전트화.
명령 수행을 위한 최적의 행동 시퀀스 자동 생성.

단일 PC 연산 병목

단일 시스템의 하드웨어 자원 한계로 인한 제어 지연 발생.
VLM/LLM 추론 시 VRAM 자원 고갈 현상.

✓ 분산 인프라 해결책

기능적 격리 및 분산 처리를 통한 초저지연 시스템 구현.
대뇌, 소뇌, 육체 시스템의 물리적 **3-PC 분산 아키텍처**.

CHAPTER 1.2

선정 배경 및 필요성

반복 작업 자동화를 넘어, 상황을 이해하여 작업을 지시하는 **Physical AI**

| AI 코딩 에이전트 도입으로 생산성 향상

코드 생성 속도

로봇 제어 코드
생성 속도

단축

60%

통합 비용

개발 및 시스템 통합
시간 비용 대비

최대
절감

75%

비용 절감

인건비 구조와 불량을 크게
낮춰 2년 이내 ROI (투자수익률)

달성
예상

100%

참고 문헌

1. Navigating the Era of Industrial Copilots in Manufacturing — automatenow.com
2. 로봇제계산학 공식 자료: Traditional robot programming vs AI & machine vision — itr.org
3. 세계경제포럼(WEF) 보고서: How AI is transforming the factory floor — weforum.org

CHAPTER 1.3

장비 및 재료

반복 작업 자동화를 넘어, 상황을 이해하여 작업을 지시하는 **Physical AI**

| 기술 스택

작업 지시 LLM AI



RTX 5080

코더 LLM, Florence AI



RTX 5080

Isaac Sim



RTX 5080

가상환경



NVIDIA

Isaac Sim & Isaac Lab

네트워크



ROS2 DDS · 웹소켓

| 춘식아 화성가자 AI 프레임워크



Isaac Sim



Isaac Lab

ROS 2

ROS2 humble/Jazzy



Python



Florence



DeepSeek



Hermes

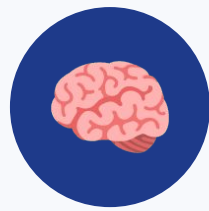


Qwen



Ollama

| 춘식아 화성가즈아 핵심 기술 3가지



강화된 AI 추론

고정식 궤적 연산 기술을 극복하여 VLA(시각·언어·행동) 모델로 탑재로 작업물의 위치나 모양이 비정형으로 변화해도 자율적인 작업 조정이 가능합니다.



AI 자동화

더 이상 정밀한 소프트웨어 엔지니어링을 요구하지 않습니다. 말(음성)을 인터랙션으로 추론 시가 이해하여 새로운 작업을 곧바로 프로그래밍합니다.



AI 강화학습

시행착오를 기반으로 최적의 행동을 학습하여 성능을 지속적으로 향상시키는 AI 기술입니다.

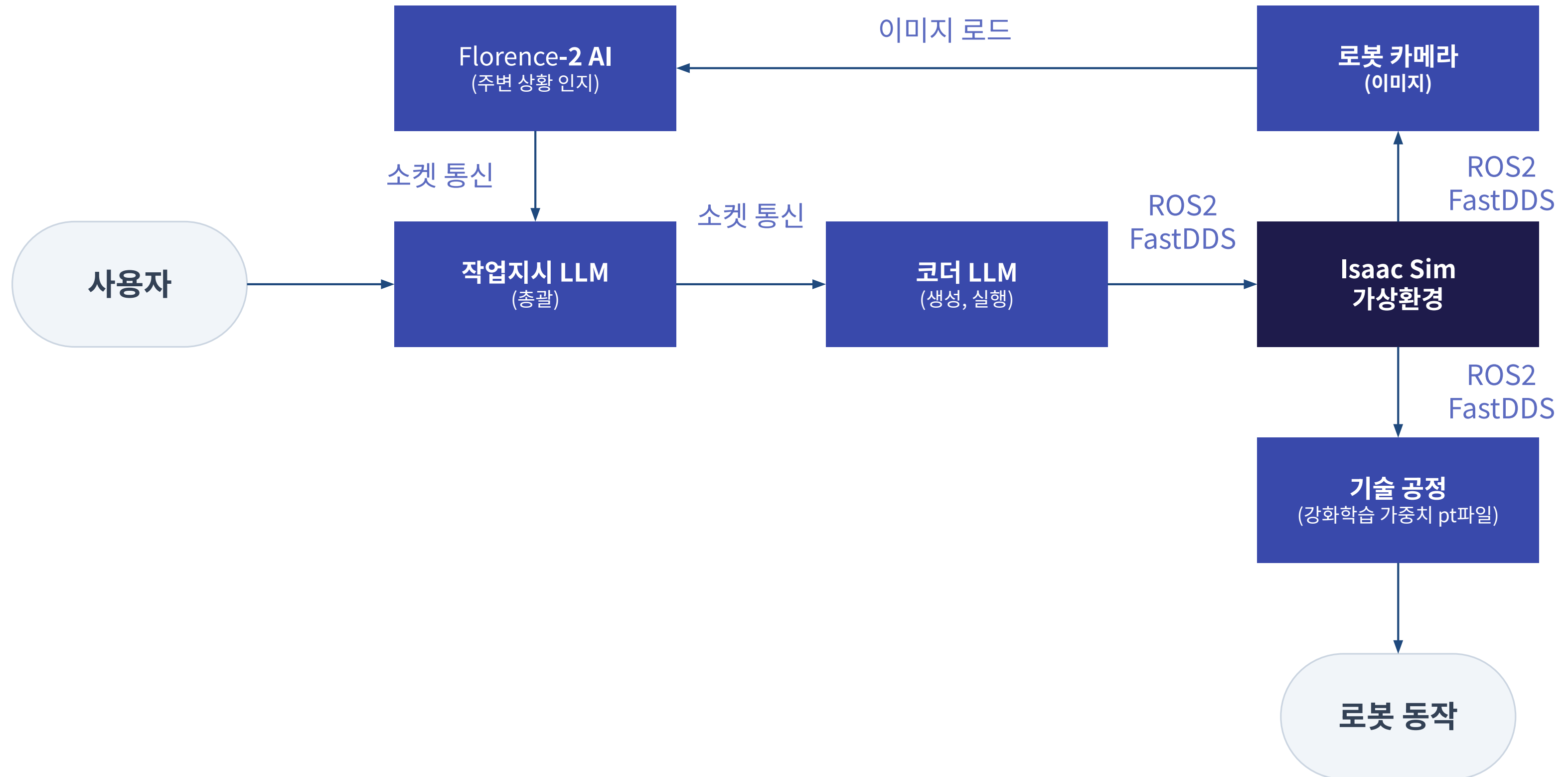
| 개발 단계별 로드맵



| 사용자 춘식이 AI - 아이작심 흐름



I 시스템 상세 아키텍처



CHAPTER 2

프로젝트 팀 구성 및 역할

단계적 통합과 검증을 통한 자율 협동 시스템 구현

I 개발 팀 구성 역할

C-3 조

성명	역할	담당 업무	상세
홍성욱	조장	시스템 통합 및 맵 제작	3PC 소켓 통신 연결, 시스템 통합 설계 및 수정, 가상 환경 맵 제작
양준호	조원	작업 지시 LLM	작업 지시 LLM, 픽애플레이스 강화학습, 가상환경 맵 제작
송종진	조원	코더 LLM	코더 LLM 및 Nav, 자세 복구 모듈 제작
최찬우	조원	강화학습	픽애플레이스 강화학습, 강화학습 기술 고도화
안교진	조원	비전 AI	Vision AI 환경인식, 4족 로봇 강화학습

CHAPTER 3

프로젝트 수행 절차 및 방법

분산된 AI가 연결되어 **하나의 지능**으로 동작

| 개발 일정

이름	담당 업무	일정9일	30일	31일	1일	2일	4일	5일	6일	7일	8일	9일	10일	11일
모두	아이디어회의													
양준호	상황인지 LLM													
송종진	스케줄링 LLM													
최찬우	강화학습													
안교진	Vision AI													
홍성욱	미니통합, 통신연결													
모두	로봇, 환경맵 제작													
양준호	상황인지 고도화													
송종진	스케줄링 고도화													
최찬우	강화학습 고도화													
안교진	Spot 강화학습													
홍성욱	전체통합													
모두	시스템 점검 / 개선													

▶ 트러블 이슈에 따라 일정은 변경될 수 있습니다.

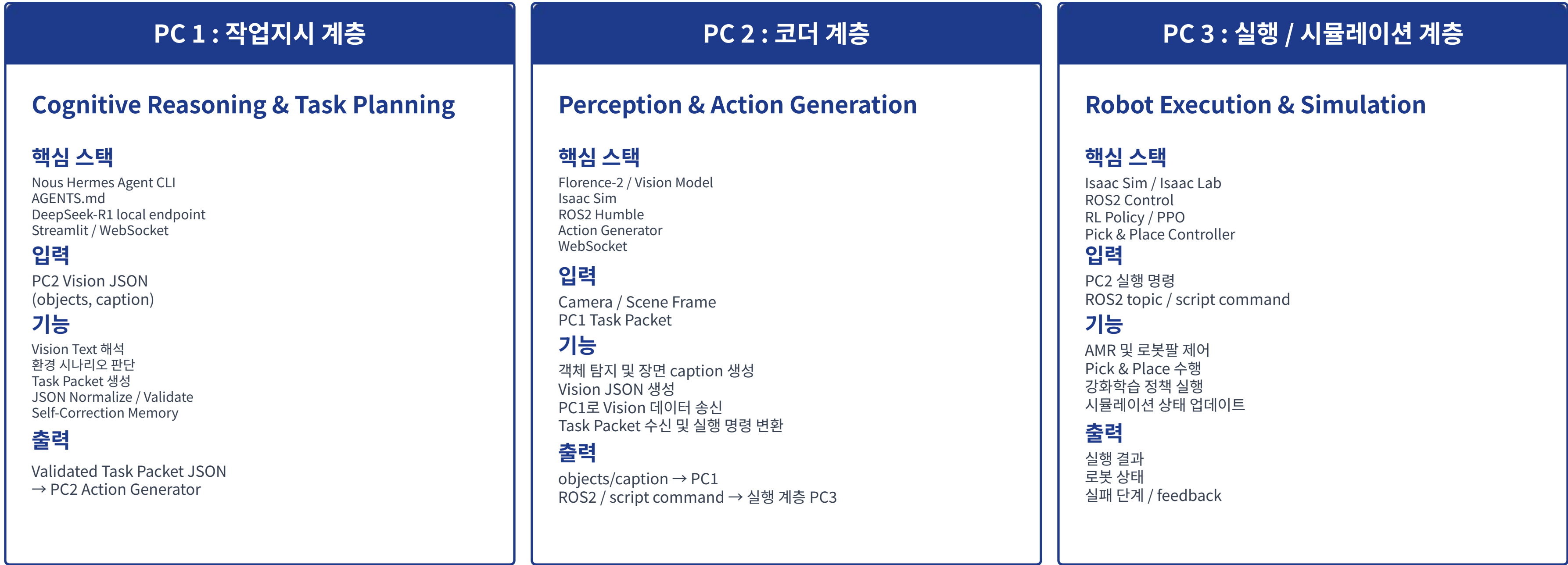
CHAPTER 4

프로젝트 수행 경과

인식과 추론, 제어를 연결하는 **Semantic AI** 기술 구조 소개

I 분산형 엣지 시스템 아키텍처

3-PC 분산형 VLA 로봇 에이전트 아키텍처



현재 구조에서는 PC2가 Vision 인식과 실행 중계를 담당하고, PC1은 Hermes Agent 기반 대뇌 계층으로서 Vision JSON을 해석한다.

| 전 계층 통합 운용 흐름

STEP 1. 코더 (PC 2)



시각 정보 추출

florence 2 모델이
환경 내 객체 태그 추출

STEP 2. 작업지시 (PC 1)



환경 맥락 인식

태그 기반으로 **최적 시나리오** 결정

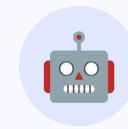
STEP 3. 코더 (PC 2)



스케줄링 및 빌드

학습 모델을 순차 배치하여
실행 스크립트로 자동 생성

STEP 4. 아이작심 (PC 3)



시뮬레이션 실행

Isaac Sim Lab 환경에서
로봇 풀러 제어 수행

↻ Closed-Loop 피드백: 실행 후 변화된 환경 재인식

● **핵심 프로세스:** 소뇌(PC 2)는 **Skill Bank** 내에 저장된 Isaac Lab 학습 모델들을 시나리오에 맞춰 순서대로 호출하는 '**자동 스케줄러**' 역할을 수행합니다.

SECTION 4.1

Vision AI

인식과 추론, 제어를 연결하는 **Semantic AI** 기술 구조 소개

| Visual AI 환경 인식

환경: Office



환경: Mars



환경: Warehouse



AMR 모양 + 사람 모양 인식 → 사물 인식



AI 추론 시스템

창고 환경
사무 환경

Hermes — 환경 추론엔진

사무 환경
화성 / 농장

환경 적합 TASK 구성

Vision AI 환경 인식 시스템은 AMR 카메라에서 촬영된 이미지를 기반으로 사물들을 인식하여 AI 추론 시스템에 전달.
Hermes가 인식된 물체들을 기반으로 환경 추측 후 환경에 적합한 TASK 구성.

| Visual AI 모델 YOLO-World

YOLO-World



▶ 효율적인 오픈 어휘 탐지

YOLO-World는 많은 자원을 소모하는 기존 Transformer 모델 대신 CNN 기반의 YOLOv8 프레임워크를 활용하여, 새로운 시나리오에도 빠르고 다양한 객체를 탐지

▶ 실시간 성능 및 비용 절감

강력한 성능을 유지하면서도 컴퓨팅 요구 사항과 비용을 크게 줄여, 즉각적인 결과가 필요한 산업 분야의 실시간 애플리케이션에 적합

▶ 프롬프트 후 탐지 (Prompt-then-detect) 전략

사용자 정의 프롬프트로 사전 계산해 오프라인 어휘 임베딩으로 저장·사용하는 방식을 도입하여, 탐지 프로세스의 효율성을 극대화

✓ 장점

압도적인 실시간 처리 속도 · '프롬프트 후 탐지'를 통한 연산 최적화

✗ 단점

사전 키워드(가중치) 입력해야 새 환경 적응 · 시각적 문맥(Context) 이해 능력 부재

| Visual AI 모델 Florence-2

Florence-2 — Microsoft's Latest Vision-Language Model



```
[10/13] mars5.jpg
=====
🔍 탐지된 물체 (1개):
- animal x1
📄 이미지 설명: The image shows a vast desert landscape with a vast expanse of red sand. The sky is a pale orange color, and the horizon is visible in the distance. The ground is covered in small rocks and boulders, and there is a large rock formation in the center of the image. The rocks are of different sizes and shapes, and some are scattered across the landscape. The overall impression is of a barren and desolate landscape.
```

▶ 효율적인 소형 멀티모달 VLM

마이크로소프트가 발표한 Florence-2는 기존 시각 언어 모델보다 크기가 작으면서도 객체 탐지, 분할, 캡셔닝, 시각적 연결 등 다양한 컴퓨터 비전 작업을 단일 통합 프레임워크로 처리

▶ 압도적인 방대량의 FLD-5B 데이터셋

1억 2,600만 장의 이미지와 54억 개의 어노테이션으로 구성된 대규모 데이터셋을 기반으로 학습되어, 광범위한 작업에서 높은 정확도와 뛰어난 효율성을 보장함

▶ 새로운 기준의 제로샷 성능

별도의 교육 없이도 작업을 즉각 수행할 수 있는 탁월한 성능을 확보하여 장시간의 훈련 필요성을 줄이고, 다양한 산업 분야에 적용 가능

✓ 장점

여러 모델을 섞을 필요 없이 단일 처리 가능 · 우월한 ZeroShot 문맥 파악력

✗ 단점

YOLO에 비해 추론 속도가 낮아 실시간 처리 불가 · 환각(Hallucination) 발생 위험성 존재

Visual AI 모델 선택 Florence-2

Florence-2



선택 이유

- 단일 모델 사용으로 파이프라인 단순화 가능
- 모델 경량화를 통한 인프라 비용 최적화
- FLD-5B 데이터셋 기반으로 사전학습되어 다양한 환경 적응가능
- 픽셀 단위 분할(Segmentation)을 통한 정밀한 환경 데이터 확보
- 자연어 기반의 고차원적 문맥이해 가능



```
// [17/18] warehouse2.jpeg 결과
{
  "objects": {
    "labels": [
      "box"
    ],
    "counts": {
      "box": 7
    }
  },
  "caption": "The image shows the interior of a large warehouse with rows of shelves. The shelves are filled with cardboard boxes of various sizes and colors. The floor is made of concrete and the ceiling is high with metal beams and lights. The warehouse appears to be well-lit with natural light coming in from the windows on the right side of the image. The image is taken from a low angle, looking down on the shelves.",
  "source": "florence2",
  "timestamp": "2026-06-10T12:16:09"
}
```

| 차후 개선 방향



NVIDIA Isaac Sim 기반 가상 로봇 제어 화면

실시간 물리 연산 런타임

▶ Isaac Sim

PC 2가 보낸 코드를 수신하여 100Hz 단위의 실시간 로봇 궤적 및 물리 제어 수행

▶ YOLO-World

Florence-2 분석자료 기반으로 가중치를 부여하여 사전학습 없이 객체 인식

▶ ROS2 Humble

내부 통신망을 통한 하위 모터 제어 및 센서 데이터 수집

SECTION 4.2

작업지시 LLM

인식과 추론, 제어를 연결하는 **Semantic AI** 기술 구조 소개

| PC1 VLA Brain Center 기능 개요

시스템 내 핵심 역할

본 모듈은 Vision 결과를 직접 로봇 제어 명령으로 바꾸는 것이 아니라, **Nous Hermes Agent**와 **AGENTS.md 규칙**을 기반으로 로봇 실행 계층이 이해할 수 있는 표준 작업 명세서 (Task Packet)로 변환하는 **PC1 대뇌 계층**입니다.

▶ 입력 PC2 Vision 데이터 (Objects, Caption)

⚙️ 처리
Nous Hermes Agent CLI + AGENTS.md 기반 추론
• Vision Text 해석 | 환경 시나리오 판단
• Task JSON 후보 생성

▶ 출력 PC2용 표준 Task Packet JSON 전송

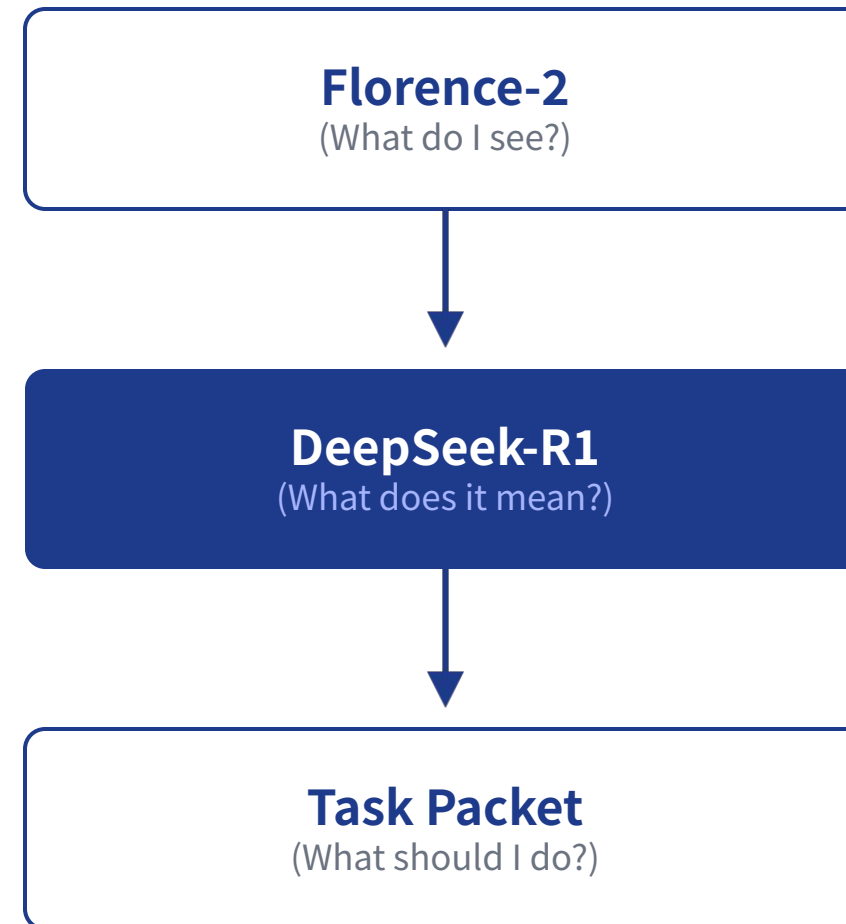


I DeepSeek-R1 선정 근거

로컬 기반 VLA Brain 구축을 위한 추론 모델 선정

시스템 요구사항

- Florence-2 Vision 결과 해석
- 환경(Context) 이해 능력
- 작업 순서 계획(Task Planning)
- JSON 구조화 출력 보장
- RTX 5080 단일 GPU 로컬 실행
- Hermes Agent 구조와 연동



DeepSeek-R1 선택 이유

1. 강력한 추론 능력

환경의 의미 해석 및 작업 생성
rock, crater → Mars
→ Collect Rock Sample

2. Task Planning 지원

Vision → 행동 계획 변환
Search → Move → Grasp

3. Agent 연동성

- AGENTS.md 프롬프트 제어
- Self-Correction 적용
- Tool Calling 확장 가능

4. 완전 로컬 운영

- 클라우드 비용 0원
- 인터넷 연결 불필요
- 보안 및 실시간성 확보

DeepSeek-R1은 Vision 결과를 맥락으로 해석하고 실행 가능한 Task Packet으로 변환하는 최적의 로컬 추론 엔진임

| vla_brain_center.py 내부 모듈 구조

AGENTS.md

Task 규칙 / 시나리오 정의

Vision Store

실시간 데이터 캐싱 관리

PC3 Receiver

WebSocket 백그라운드 서버

Hermes Core

Hermes CLI + AGENTS.md 추론

Packet Guardrail

JSON 정규화 / 타입 검증

PC2 Sender

검증 패킷 전송 + 중복 차단

Log Memory

성공/실패 기록 관리

Control UI

Streamlit 통합 관제 대시보드

본 코드는 **UI · 통신 · Hermes CLI 추론 · JSON 검증 · 로그 기록 · PC2 전송**을 하나의 **Brain Center** 모듈로 묶어 관리합니다.

| PC2 Vision 데이터 수신 구조



기술 핵심

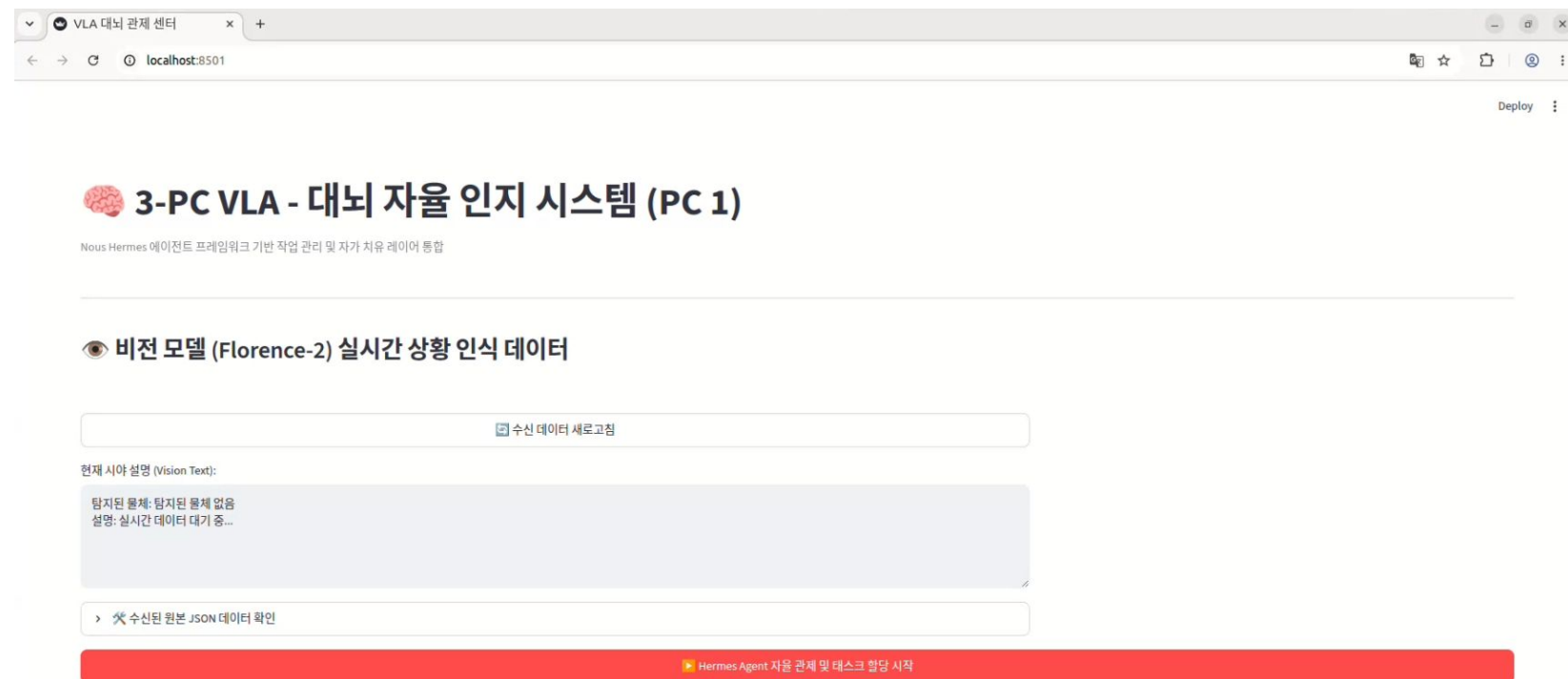
websockets 라이브러리로 서버를 구동하며, **asyncio**와 **threading**을 이용해 UI와 독립된 백그라운드 스레드에서 실행됩니다.

수신 데이터 포맷

```
{
  "objects": [{"counts": {"box": 2, "conveyors": 1,
    "qr_marker": 1}}],
  "caption": "A warehouse scene with boxes and conveyor"
}
```

objects: 탐지 객체 정보 | caption: 장면 요약 문장
→ GLOBAL_VISION_DATA 저장 → Vision Text로 변환
→ Hermes CLI Prompt 입력으로 사용

| Streamlit 기반 실시간 관제 UI



Human-in-the-loop 관제

- 수신 데이터 표시** 탐지 객체 목록 및 장면 캡션 실시간 갱신
- 상태 유지** `st.cache_resource`로 데이터 및 서버 스레드 보존
- 자동 갱신** `st.autorefresh`로 2초마다 화면 동기화
- 실행 제어** 분석 시작 버튼 및 실시간 처리 로그 시각화
- 중복 실행 방지** `run_requested / is_analyzing` 상태값 관리
- 중복 전송 방지** `last_sent_payload_hash` 해시값 검증

단순 화면 표시를 넘어, **입력 확인 → Hermes CLI 실행 → Guardrail 검증 → PC2 전송 결과 확인**을 담당하는 인터페이스입니다.

| Hermes Agent: AGENT.md 기반 환경 분류 및 Task Packet 생성 구조

Vision 결과를 Hermes CLI로 해석하고, 실행 가능한 표준 JSON 작업 패킷으로 변환

Hermes Agent Layer

Nous Hermes CLI

Vision Text와 AGENTS.md 규칙을 입력받아 환경 단위를 해석하고 Task JSON 후보를 생성하는 Agent 추론 백엔드.



PC2 Action Generator

검증된 Task Packet을 수신하여, ROS2 / Isaac Sim / 로봇 제어 계층으로 실행을 전달하는 하위 실행 모듈.

Vision Text → Reasoning → Task JSON

01. Hermes CLI 연동

Python subprocess를 통해 'hermes -z' 명령을 비대화형으로 호출하고, AGENTS.md 규칙 기반 추론을 수행함.

02. Vision 기반 환경 시나리오 매핑

PC3에서 수신한 objects/caption을 분석하여 warehouse / farm / mars 중 적절한 작업 시나리오로 변환함.

| Nous Hermes Agnet CLI 연동 및 현재 적용 범위

Nous Hermes Agent CLI를 PC1 Brain Center의 추론 백엔드로 연동하고, AGENTS.md 기반 규칙 주입과 PC1 Guardrail을 결합하여 Task Packet 생성 구조도를 구현했습니다.



도입 배경

- LLM 출력이 항상 안정적인 JSON을 보장하지 않음 (환경 발산 가능성)
- 실패한 출력 패턴이 다음 실행에 반영되기 어려운 기존 구조의 한계



현재 구현 내용

- Hermes CLI 호출부를 **IntegratedHermesAgent** 클래스로 캡슐화
- AGENTS.md를 로드하여 Agent 규칙으로 주입
- subprocess로 'hermes -z' 비대화형 호출
- 출력 데이터 정규화(Normalize) 및 검증(Validate) 적용
- 실패 로그를 **memory log**에 기록하고 다음 prompt에 미드백 반영
- 최대 3회 Self-Correction 재시도 루프 구축



향후 연동 방향

- 현재 Hermes CLI 연동 구조를 **Hermes Skill / MCP Tool** 단위로 모듈화
- PC2 실제 물리 실행 결과(Feedback)를 Memory Layer에 직접 연결
- 반복학습 실패 양상을 분석하여 Skill 자체를 개선하는 파이프 시스템
- **MCP Tool** 형태의 확장을 통해 PC2 전송 기능을 표준 인터페이스로 통합

| Hermes CLI Self-Correction: Memory Log 기반 출력 안정화

Hermes Agent 출력이 JSON 파싱/검증에 실패하면 오류를 memory log에 저장하고, 다음 Hermes 호출 시 최근 오류 기록을 prompt에 반영하여 동일한 JSON 오류를 회피합니다.

Vision Text 입력 → Hermes CLI 호출 → Task Packet 후보 JSON 파싱/검증 → → PC2 전송 (성공)



오류 기억

01

JSON 파싱 실패, 필수 key 누락, target 필드
무 등 LLM 출력 단계의 결함 데이터를
pc1_parse_errors.json에 실시간 기록
및 영구 저장합니다.

자가보전 재추론

02

파싱/검증 실패 시 직전 prompt buffer에
추정되어 동일한 형식 오류를 반복하지 않도록
유도, 최대 3회 Self-Correction을 수행합니다.

확장 가능성

03

PC2 실행 feedback 연동 시, 실제 실행 실패
단계까지 포함하는 Task-level
Self-Correction으로 확장 가능합니다.

Hermes-style Self-Correction은 LLM이 생성한 Task Packet 후보를 곧바로 전송하지 않고 명시적 구문 검증을 거침으로써 하드웨어 레이어의 안정성을 강력하게 확보합니다.

| Hermes 출력 우선 + 템플릿 Fallback 구조

Environment	Target	Sequence 예시	역할
warehouse	box	search_qr → move_to_box → suction_grasp → release	물류 상자 이송
farm	crop	scan_crops → move_to_plant → cut_crop → release	작물 수확
mars	rock	scan_rock → move_to_rock → grasp_sample → insert	암석 샘플 채취

Hermes Agent는 **AGENTS.md** 규칙을 바탕으로 **Task Packet** 후보를 생성하며, PC1이 누락되거나 형식이 깨진 필드에 한해 **ENV_TEMPLATES**를 **fallback**으로 사용합니다.

이를 통해 **Agent 출력의 유연성**을 유지하면서도 **기본 안정성**을 확보합니다.

| LLM Output Guardrail: Normalize



- ✓ **필수 Key:** environment, task, target, pos, sequence 등 누락 확인
- ✓ **형식 검증:** 좌표 [x, y, z] 숫자 배열, sequence 문자열 리스트 확인
- ✓ **Fallback 보장:** 누락/깨진 필드는 ENV_TEMPLATES 기본값으로 보정

본 모듈은 **Hermes 출력**을 로봇 실행에 바로 연결하지 않고,

정규화와 타입 검증을 거친 **Task Packet**만 PC2로 전달합니다.

| Lightweight Memory: 성공/실패 로그 저장 구조

자가 치유 루프 (Self-Correction)

검증 실패 시 오류 원인을 prompt에 즉시 추가하여,
최대 3회 Hermes 재호출을 수행합니다.

예:

"직전 출력이 실패했다. 원인: JSON object를 찾을 수 없음.
오직 JSON object 하나만 다시 출력하라."

로그 파일 구조

로그 파일

목적

pc1_parse_errors.json

JSON 파싱/검증 오류 기록

pc1_success_packets.json

검증 성공 Task Packet 이력 저장

실패 이력은 다음 Hermes 호출 시 prompt context로 반영되며,
성공 패킷은 추후 분석 및 이력 관리용으로 저장됩니다.

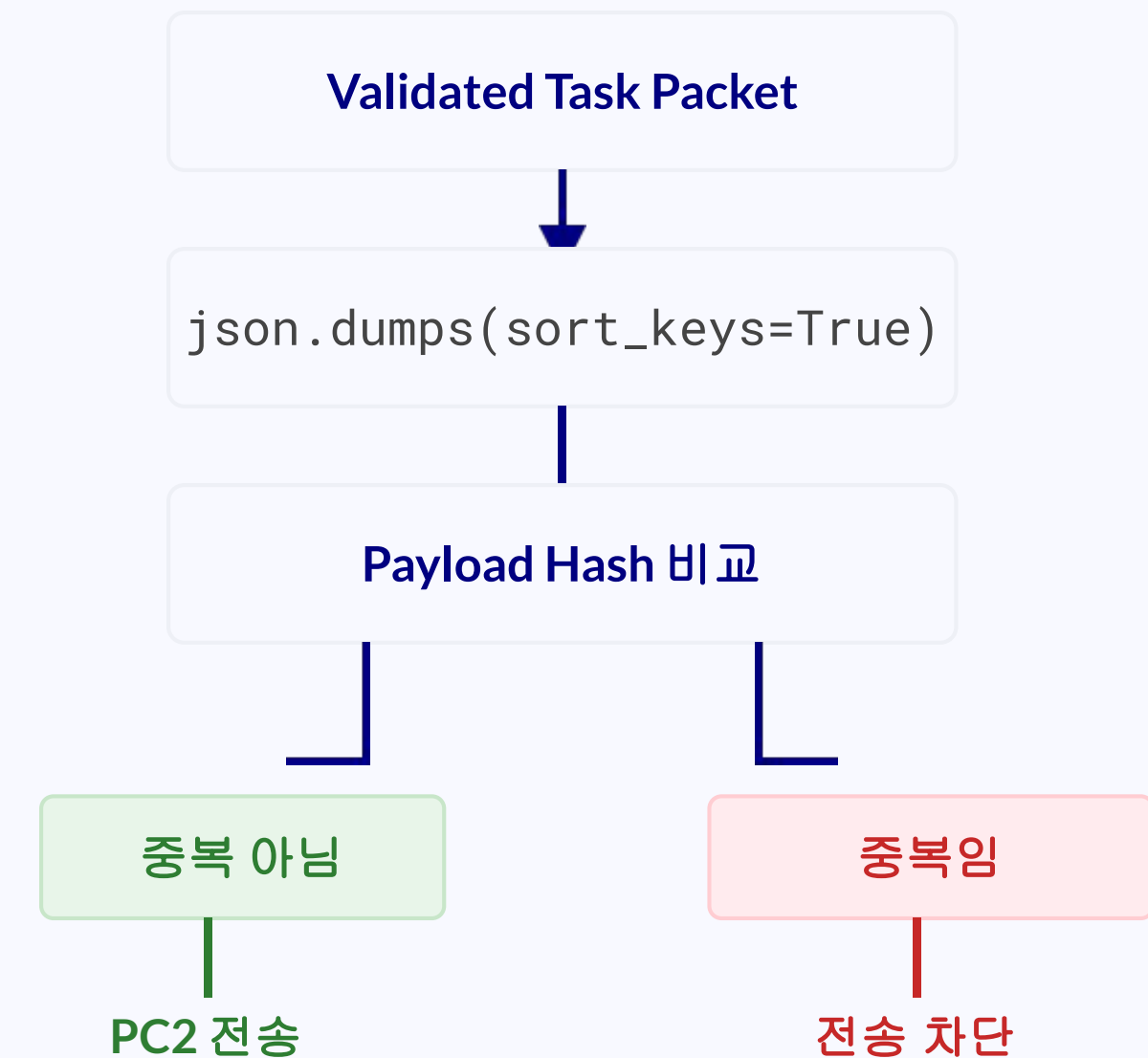
| Hermes Tool: Validated Task Packet 전송 및 중복 차단

최종 패킷 발행 (Publishing)

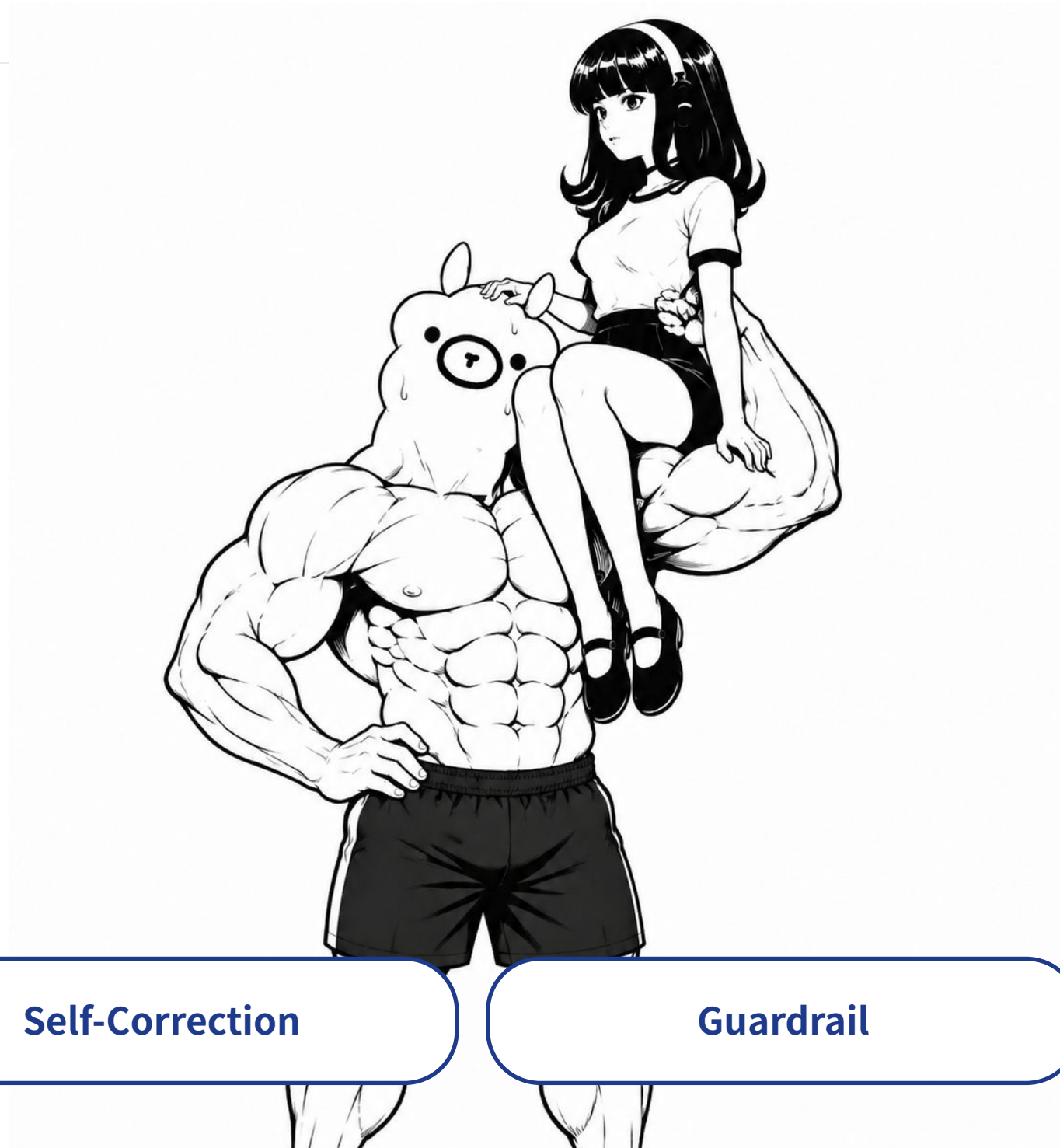
검증이 완료된 Task Packet은 `hermes_tool_send_to_pc2()` 함수를 통해 PC2 Action Generator로 전송됩니다.

- 🚩 **Target:** ws://192.168.10.36:9999
- ↔ **Method:** websocket-client create_connection() 전송
- ✓ **Safety:** Guardrail 통과 JSON만 전송
- ⊘ **Duplicate Block:** 직전 payload hash와 동일하면 중복 전송 차단

Transmission Logic Flow



| 강해진 춘식이



Memory Log

Self-Correction

Guardrail

Duplicate Block

한층 더 강력해진 자가 보정 능력

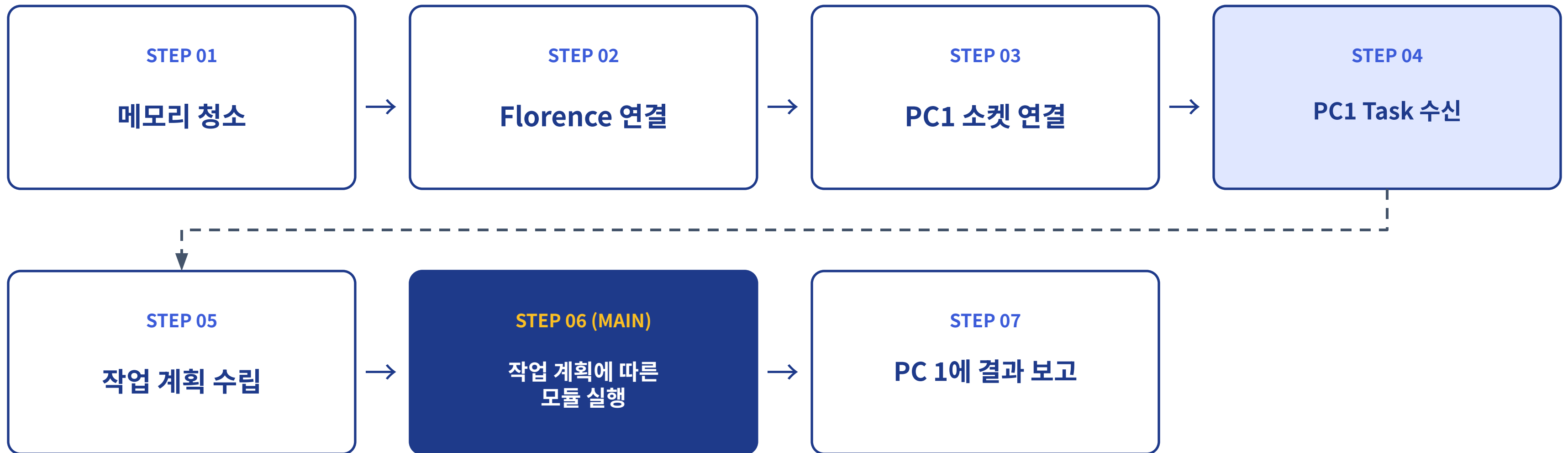
Hermes 출력이 흔들려도 4중 안전장치로 시스템 전체가 안정적으로 동작합니다.

SECTION 4.3

코더 LLM

인식과 추론, 제어를 연결하는 **Semantic AI** 기술 구조 소개

| PC 2: 학습 모델 스케줄링 (소뇌 계층)



| PC 2: 학습 모델 스케줄링 (소뇌 계층)

qwen-coder:30B

모델	걸린 시간	명령 task 개수
qwen3.5:4b	37.036초	6 스텝
qwen3.5:9b	66.893초	7 스텝
qwen3-coder:30b ★	11.069초	7 스텝

✓ 더 빠르고, 더 세세한 task를 만든 **qwen3-coder:30b**를 선택

| PC 2: 학습 모델 스케줄링 (소뇌 계층)

⚠ 하드웨어 제약 사항

- qwen-coder:30B은 그래픽 메모리 최소 **18Gb** 필요
- rtx 5080 실사용 메모리 총량은 **15Gb** 정도

⚠ 메모리 누수 및 성능 저하

- 초과된 메모리는 메모리 카드로 이전
- 캐시 형태의 정보가 메모리를 계속 점유
- 메모리 포화 시 AI 성능이 급격히 저하됨

해결 방안 및 결과

AI 동작 전 강제 메모리 청소 수행 → **항상 동일한 퀄리티의 결과 도출 가능**

| PC 2: 학습 모델 스케줄링 (소뇌 계층)

동작 순서 구성

프롬프트에 사용 가능한 동작들을 제시한 뒤 AI에게 최적의 동작 배치를 수행하도록 요청

PICK

camera로 **aruco mark**를 확인하여 대상 상자를 정확히 파지

MOVE

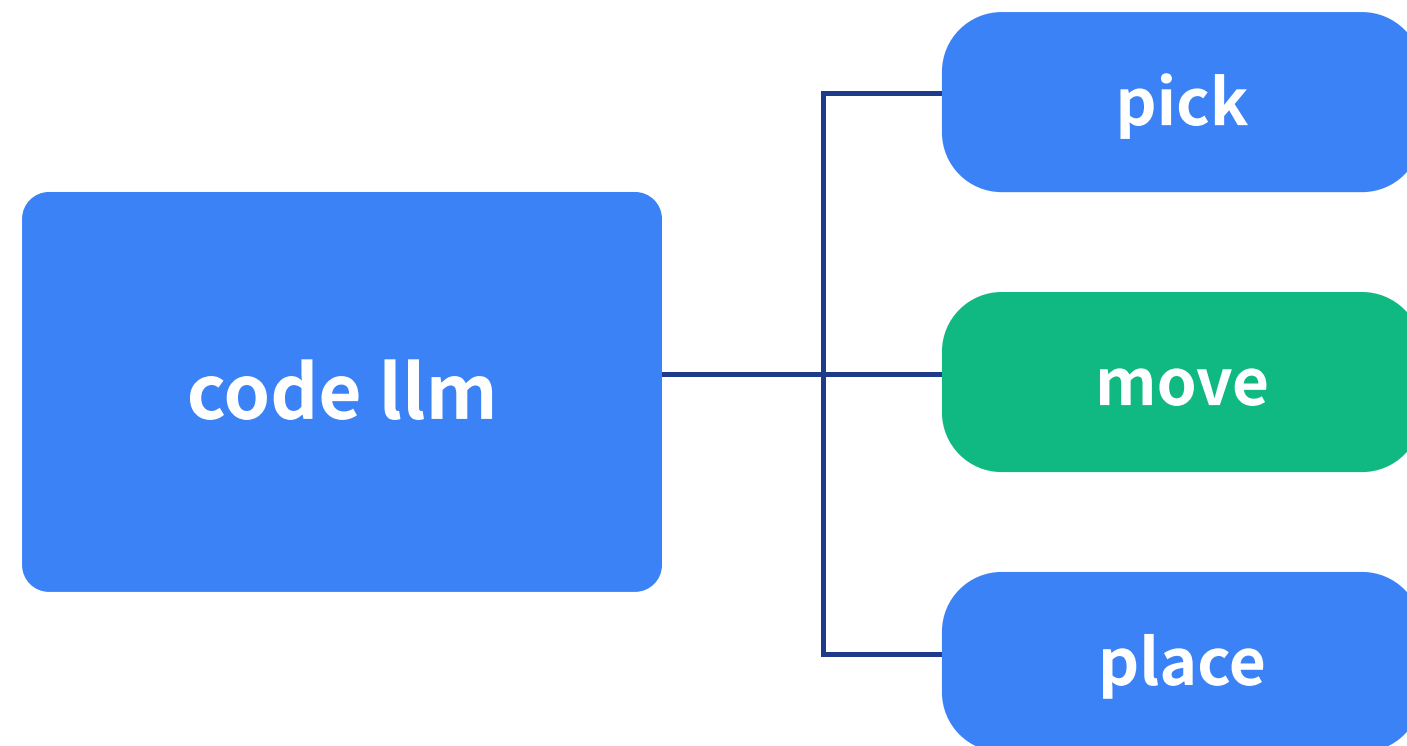
Navigation(nav) 시스템을 이용하여 지정된 특정 좌표로 안전하게 이동

PLACE

목적지 도착 후 파지 중인 물건을 **지정된 위치**에 내려놓음

| PC 2: 학습 모델 스케줄링 (소뇌 계층)

동작 모듈화 진행



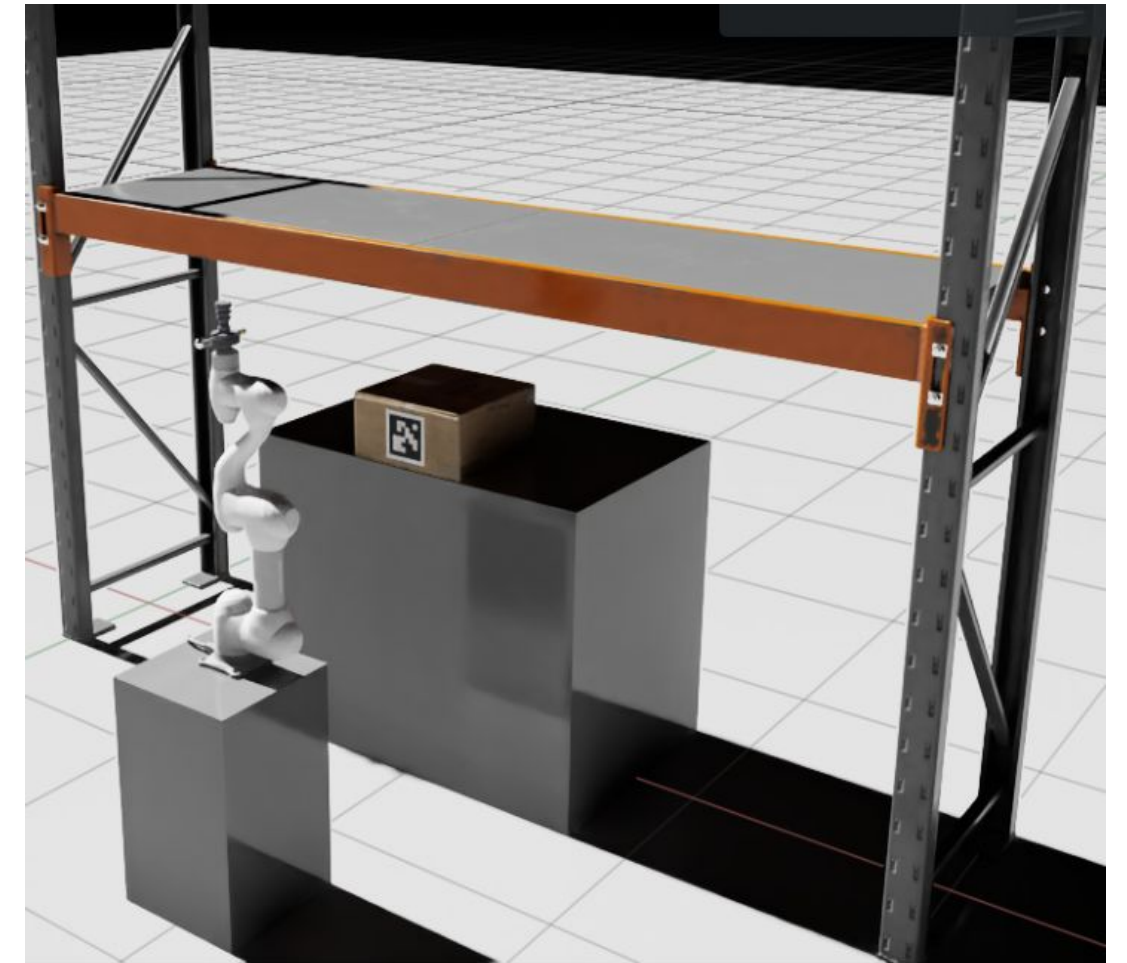
모듈화의 장점

- 동작 추가 및 수정의 용이성
- 코드 디버깅 효율성 증대
- 메인 코드 중심의 통합 제어
- Thread 활용을 통한 자원 최적화

| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

사용 도구: M0609, 흡착그리퍼, Realsense 카메라, Aruco마커를 부착한 박스

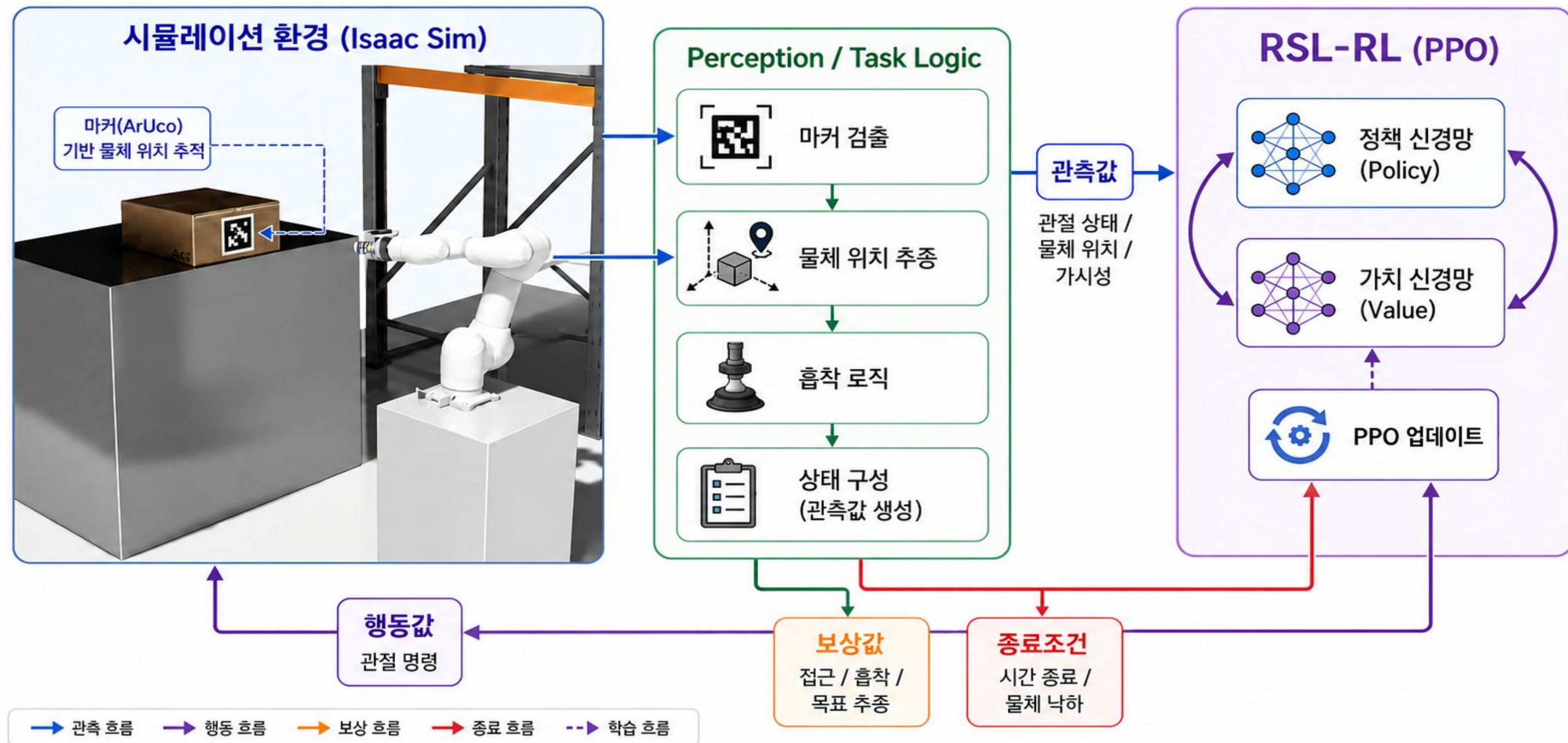


박스 pick에 최적화된 **흡착 그리퍼** 채용, 물체의 위치 추종을 위해 **Aruco 마커** 및 **RGB-D 카메라** 채용

| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

강화학습 기반 Pick & Place 시스템 아키텍처

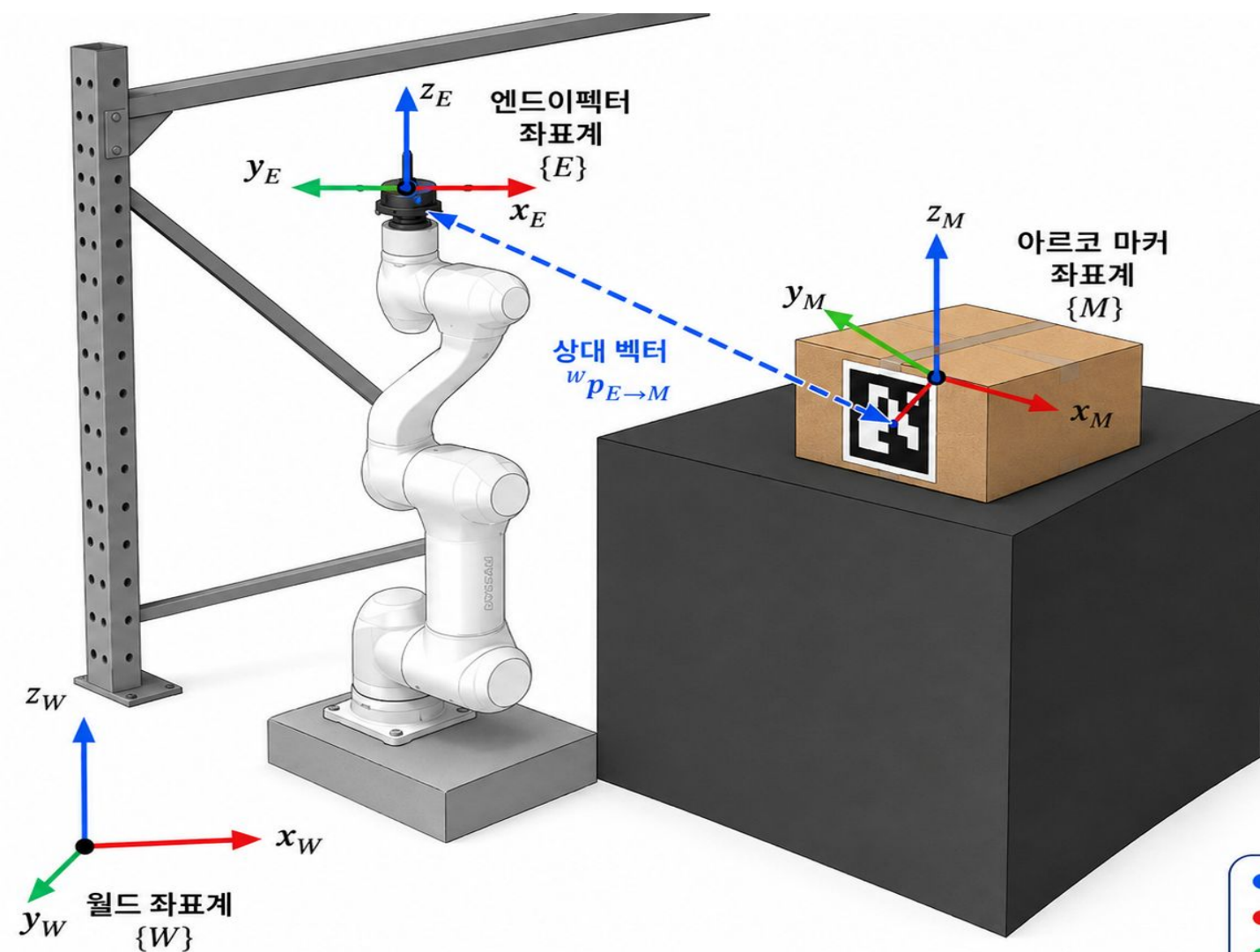


| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

카메라 추종 물체 위치 변환 로직

카메라 → ArUco → 월드 좌표계 변환



▶ 마커 기반 위치 추정

→ ArUco 중심 픽셀 좌표를 3D 위치로 복원 후 월드 좌표계로 변환

▶ EE-마커 상대벡터 계산

→ 월드 기준 두 위치의 차분으로 상대좌표 및 거리 추출

▶ Last Valid Pose Buffer

→ QR/ArUco 미검출 시 최근 유효 pose를 재사용

| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

인식한 아르코 마커 좌표 추출 및 오차 비교



검증 항목	결과
ArUco 중심 2D 좌표	u=550.75, v=333.75
카메라 기준 3D 위치	(-0.0679, -0.0187, 0.4821) m
월드 기준 3D 위치	(-1.9990, 0.5016, 1.3210) m
엔드이펙터-마커 상대좌표	(0.0400, -0.3760, 0.0568)
상대거리	0.3823 m
GT 위치 오차	0.0010 m
ArUco 중심 픽셀 좌표와 Depth 정보를 이용해 카메라 기준 3D 위치를 복원하고, 이를 월드 좌표로 변환한 결과 GT 대비 약 1.0 mm 오차로 검증되었습니다.	

| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

박스 흡착 로직



ArUco 마커와 suction_tcp 간 거리가 임계값 이내일 때 흡착을 판정하고, 흡착 순간의 상대 pose를 유지해 매 step 물체 위치를 갱신

흡착 거리 감시

→ suction_tcp와 ArUco 마커 중심 거리 계산

Attach 판정

→ threshold 이내 진입 시 흡착 상태 활성화

상대 Pose 저장

→ 흡착 순간의 object 위치·자세를 EE 기준 offset으로 기록

Pose Slaving

→ attached 상태 동안 object root를 suction_tcp 기준으로 매 step 갱신

속도 제거

→ 흡착 중 object velocity를 0으로 설정해 물리적 흔들림 완화

| PC 2: 모듈 - Pick & Place

1. 박스 Pick 학습 과정

목표 설정: [물체를 옆에서 잡고 안전하게 빼서 들어올려라]

▶ 관측값

관절 위치·속도, EE 기준 물체 위치, 물체 가시성, 목표 관절 오차, 이전 action

▶ 행동값

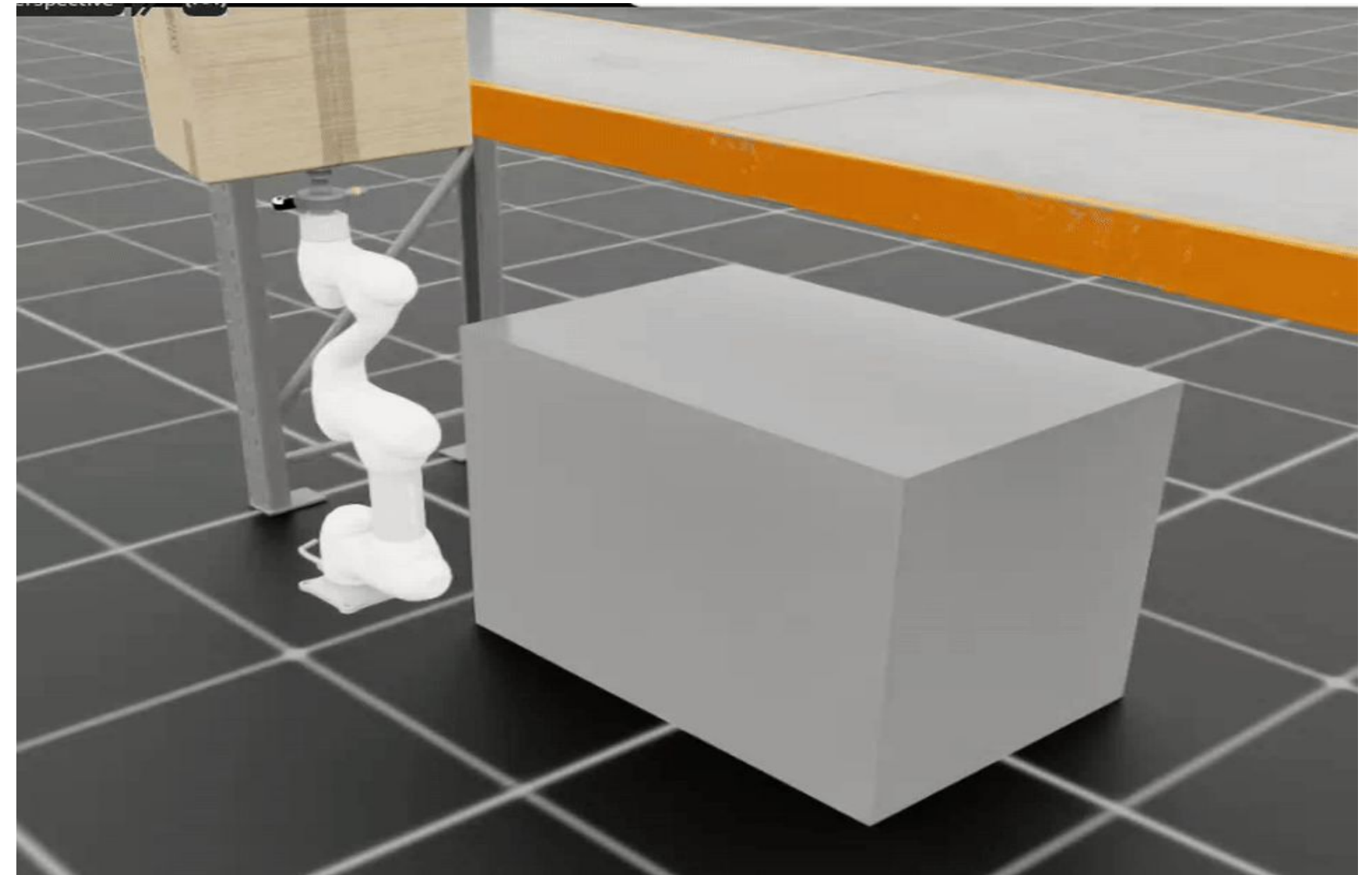
6축 로봇팔 관절 위치 명령 / ※ 흡착 action이 아닌 suction 로직으로 처리

▶ 보상 및 패널티

접근·흡착·목표 자세 추종 보상 / EE 저하·력 충돌·급격한 action·관절 속도 패널티

▶ 커리큘럼

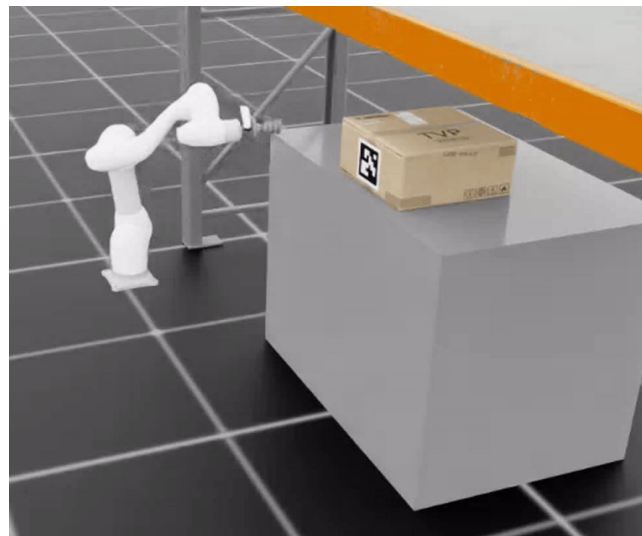
보상 가중치를 단계적으로 조정하여 초기 접근 / 흡착 학습 후 안정적인 관절 움직임 학습



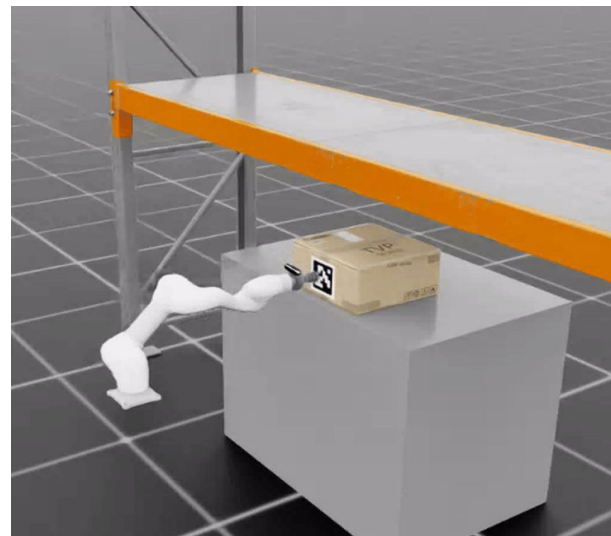
정책은 관절 상태와 ArUco 기반 물체 위치를 관측하여 6축 관절 명령을 출력하며, 보상·패널티와 커리큘럼을 통해 흡착 성공뿐 아니라 충돌 방지와 안정적인 동작까지 함께 학습하도록 구성했습니다.

| PC 2: 모듈 - Pick & Place

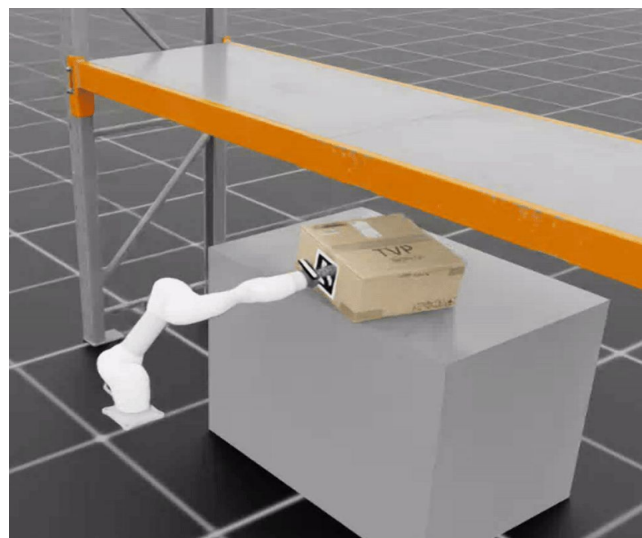
2. 박스 Pick 학습 과정



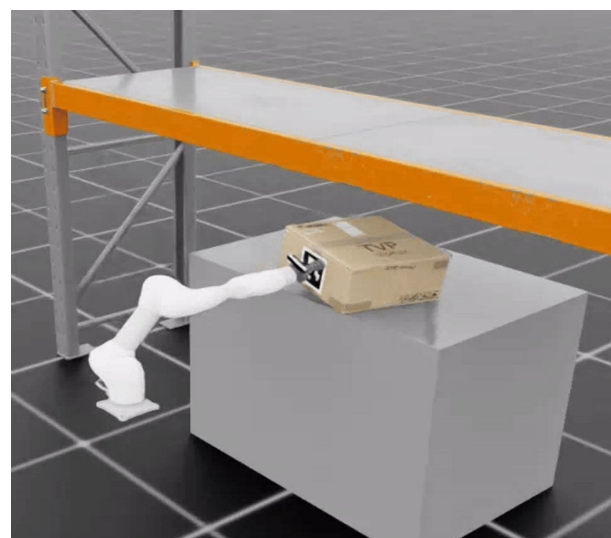
학습 초반 (iteration: 1)



학습 중반 (iteration: 13,200)



학습 후반 (iteration: 21,300)



학습 완료 Play

Pick 학습 단계

초반: 접근 행동 학습

ArUco 마커에 가까워질수록 보상을 받아, 물체 방향으로 이동하는 행동을 학습

중반: 흡착 및 리프팅 학습

박스를 흡착으로 들어올리면 보상을 받아, Pick 수행에 필요한 동작을 학습

후반: 안정화 동작 학습

action rate와 관절 속도 패널티를 통해 떨림을 줄이고, 부드러운 관절 움직임을 학습

| PC 2: 모듈 - Pick & Place

2. 박스 Place 학습 과정

목표 설정: [들고 있는 물체를 목표지점에 갖다놔라]

▶ 관측값

관절 위치·속도, 물체 위치, 목표 좌표, 이전 action

▶ 행동값

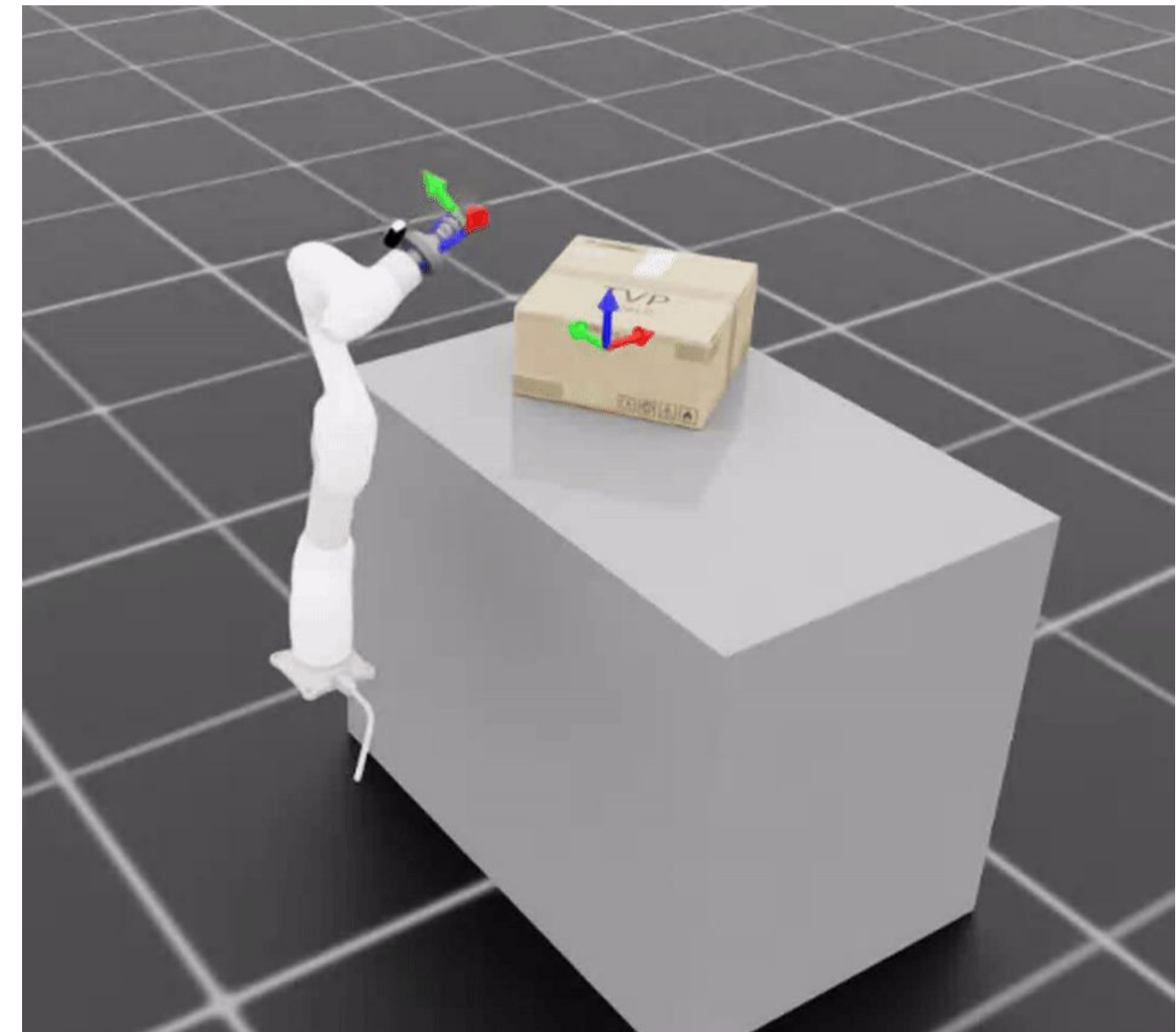
6축 로봇팔 관절 위치 명령

▶ 보상 및 패널티

목표 접근, XY 정렬, 목표 근처 release, 안정적 배치 보상 / 조기 낙하, 물체 낙하, 급격한 action, 높은 관절 속도 억제

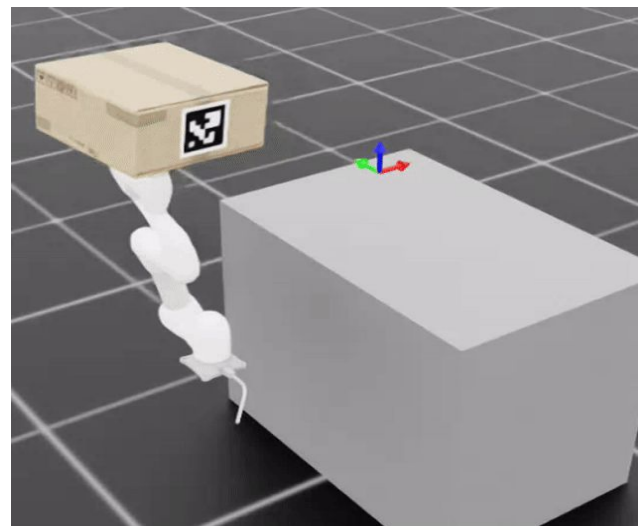
▶ 커리큘럼

학습 후반으로 갈수록 안정성 관련 패널티를 강화하여 부드러운 Place 동작 유도

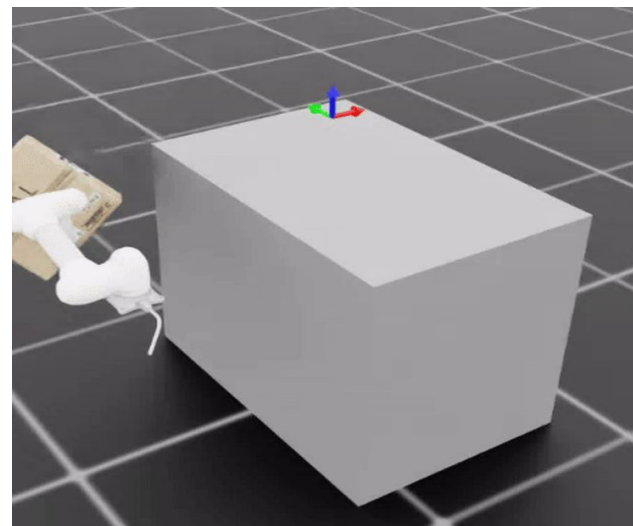


| PC 2: 모듈 - Pick & Place

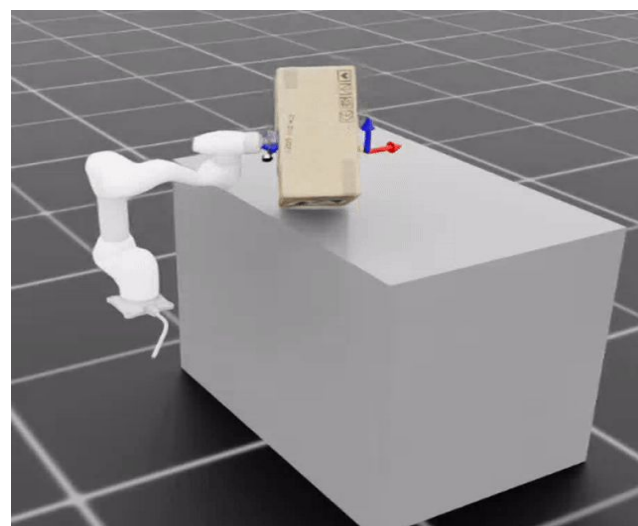
1. 박스 Place 학습 과정



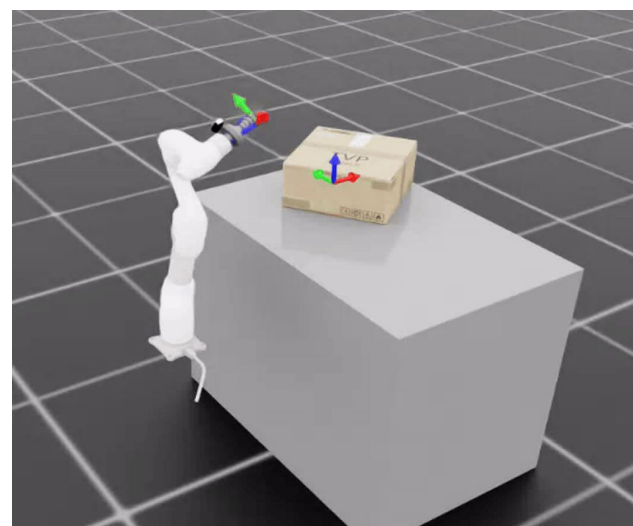
학습 초반 (iteration: 1)



학습 중반 (iteration: 1,000)



학습 후반 (iteration: 2,000)



학습 완료 Play

Place 학습 단계

초반: 목표 접근 학습

박스가 목표 지점에 가까워질수록 보상을 받아, 목표 방향으로 이동하는 행동을 학습

중반: 빠른 목표 이동 학습

목표 지점까지 더 빠르게 이동할수록 보상을 받아, 빠르게 이동하는 정책을 학습

후반: 정밀 배치 및 흡착 해제 학습

목표 근처에 정교하게 도달한 뒤 흡착 해제에 성공하면 보상을 받아, 안정적인 Place 동작을 학습

| PC 2: 모듈 - Pick & Place

3. 개선사항



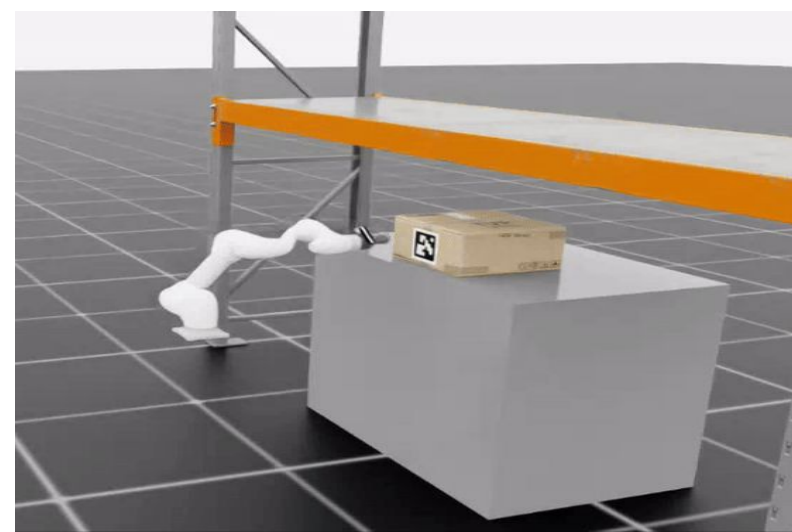
학습 완전 실패

Pick 실패 원인

ArUco 3D 좌표 오차와 박스 인식 여부를 구분하지 않는 관측값 부재로 인해, 정책이 잘못된 물체 위치를 기준으로 접근함

개선 사항

ArUco 좌표를 GT 기준으로 보정하고, `object_visible` 관측값을 추가하여 인식/미인식 상태를 판단하도록 개선



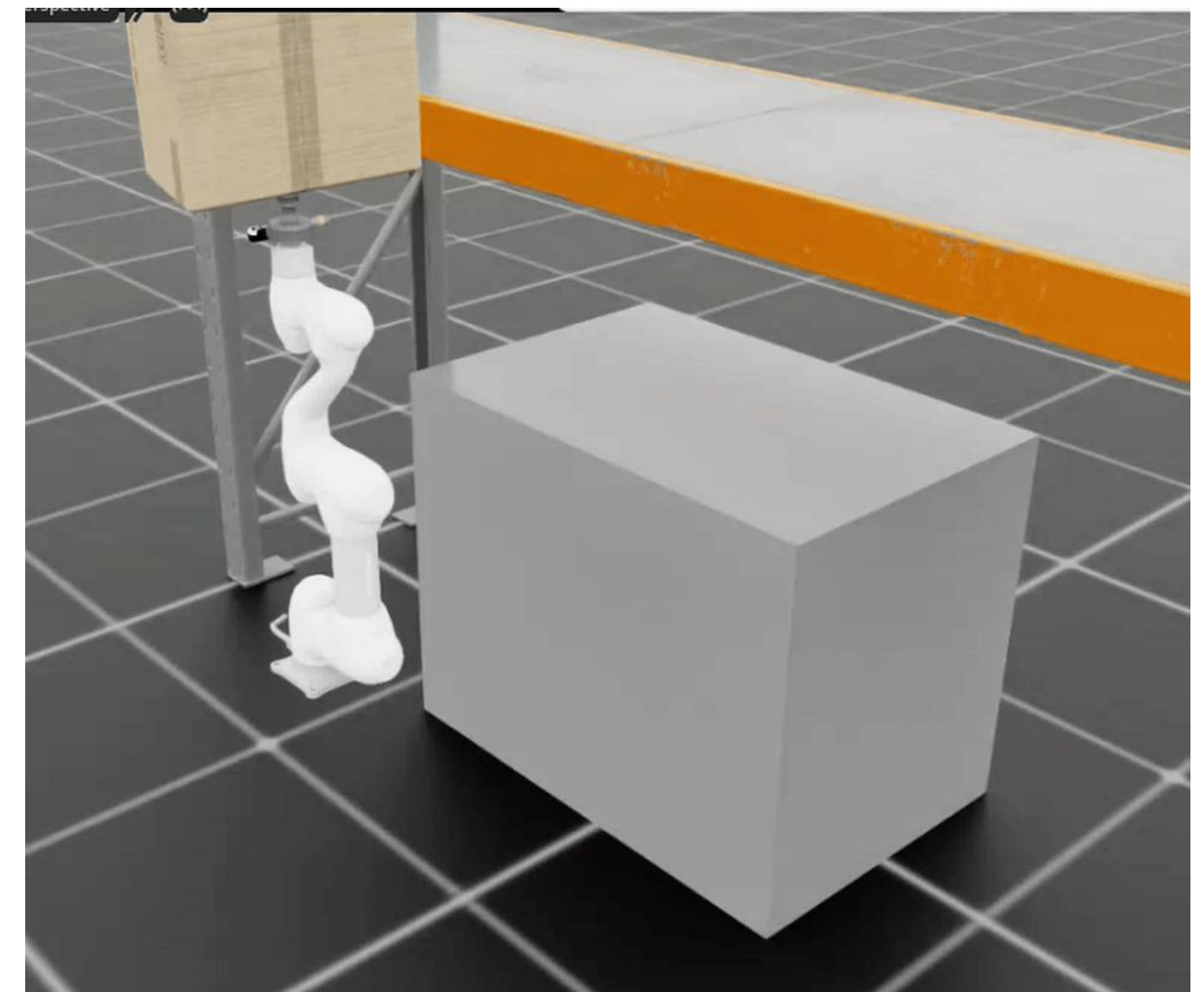
부자연스러운 흡착

흡착 로직 문제

단순 거리 기반 흡착으로 부착 순간 물체 pose가 튀거나 상대 위치가 어긋남

개선 사항

ArUco 마커 GT 기준으로 흡착을 판정하고, 흡착 순간의 EE 기준 offset을 유지해 매 step 물체 pose를 `suction_tcp`에 종속

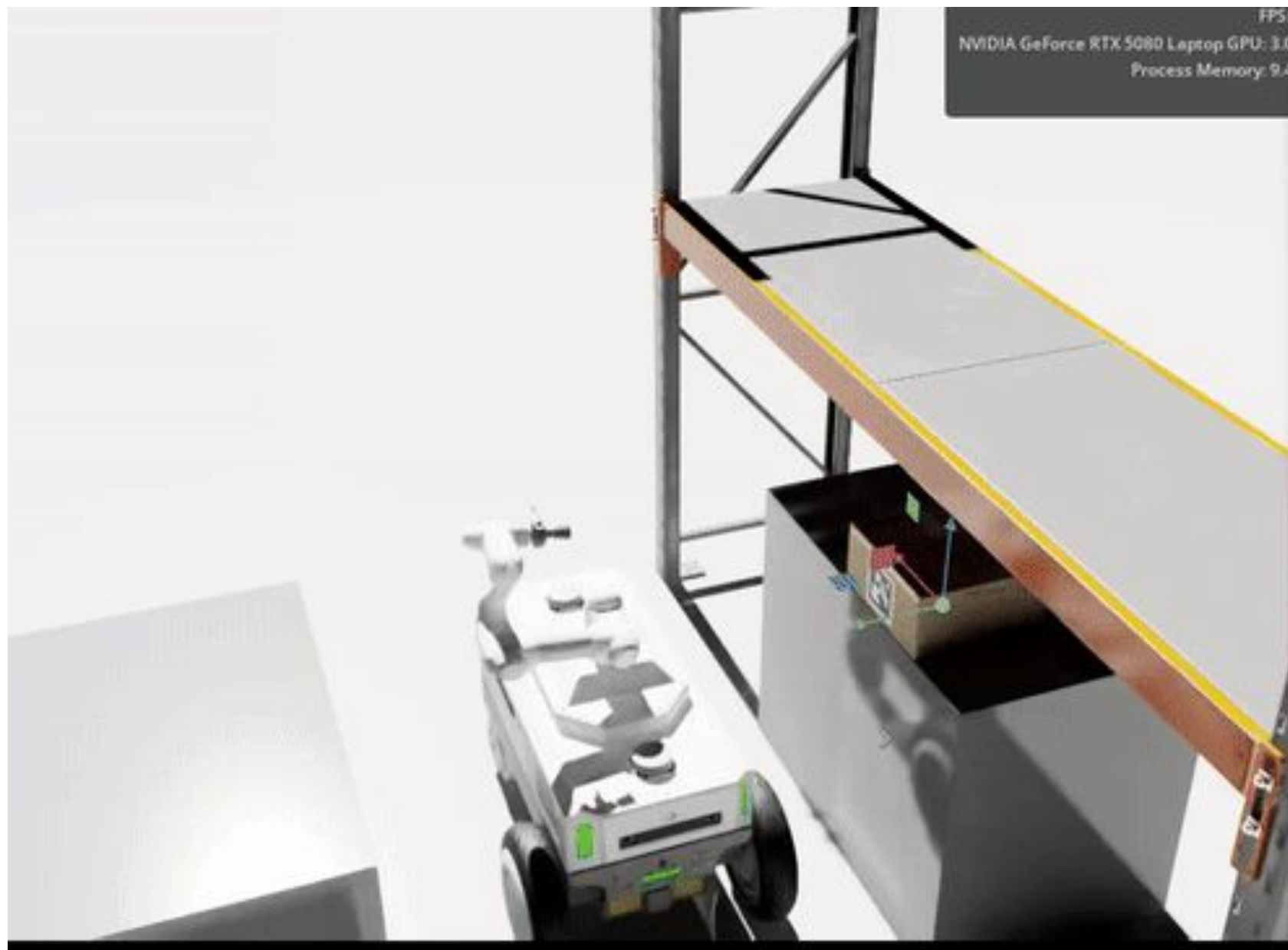


자연스러운 모션으로 Pick Task 성공

| PC 2: 모듈 - Pick & Place

3. 개선이 더 필요한 사항

모델을 사용하여 Pick을 수행할 경우 로봇이 넘어지거나 크게 흔들리는 현상 발생



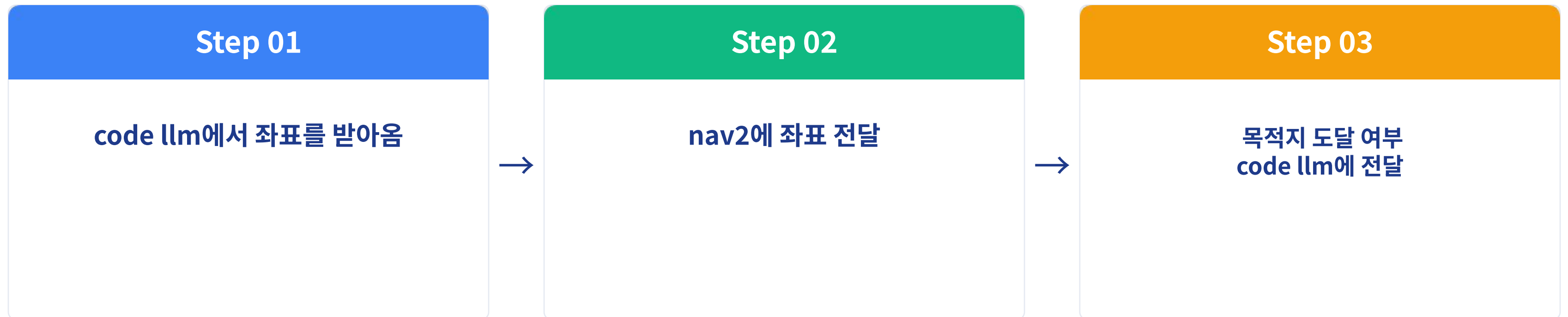
⚠ 문제점

로봇팔의 관절 움직임이 과도하게 빠르고 불안정하게 발생하여, AMR 위에 장착된 로봇의 무게중심이 흔들리고 전체 플랫폼 안정성이 저하됨

▶ 개선 방향

관절 속도와 action 변화량에 대한 패널티를 강화하여 부드러운 움직임을 유도하고, 고정 베이스가 아닌 **AMR 탑재 환경**을 반영한 학습 조건에서 정책을 재학습해야 함

| PC 2: 모듈 - move



① 추가 고려 사항

- 넘어졌을 때를 고려해 자세 복구 예외 처리 추가

| PC 2: 모듈 - move

⚠ 문제점 : Carter에 collision 적용 시 동작이 이상해짐

원인 분석 및 해결 방안

1. 충돌 현상 파악

- collision 적용 시 뒷바퀴가 굳는 현상 발생
- carter base link와 뒷바퀴 간 충돌 확인

2. 검증 및 조치

- collision 선택시 물리적 간섭 확인

→ collision preset 제거로 해결



| PC 2: 모듈 - move

⚠ 문제점 : 라이다 센서와 맵의 불일치

Parameter 조정 및 정밀도 향상

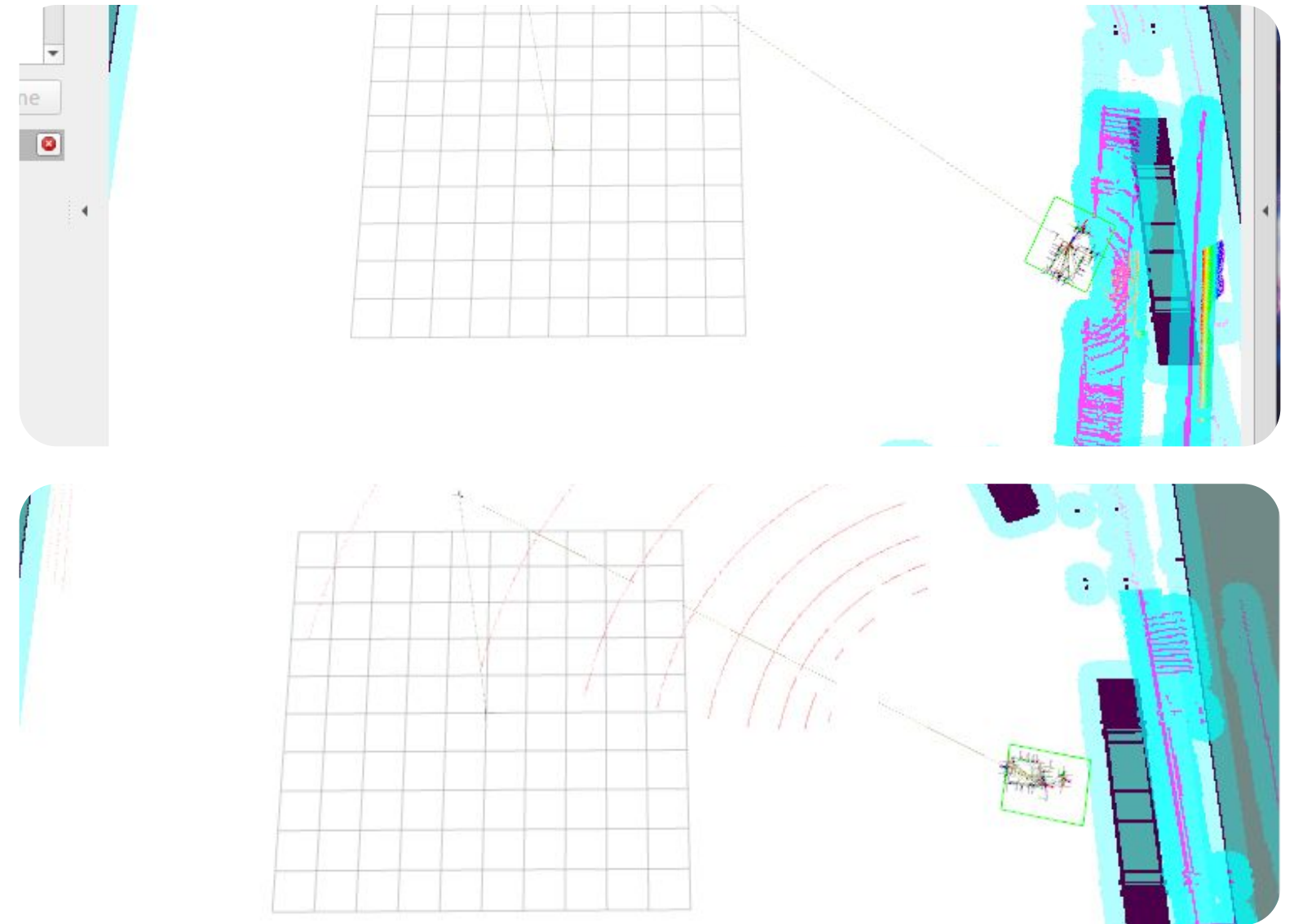
1. update_min_d: 0.25 → 0.05

● 로봇의 위치 좌표 갱신 주기 감소

2. update_min_a: 0.2 → 0.02

● 로봇의 각도 갱신 주기 감소

→ 위치 정밀도 향상 완료



2D 라이다 스캔 시각화 (조정 후 정합)

| PC 2: 모듈 - move

⚠ 문제점 : Y 축의 좌표만 정확하지 않음

Parameter 수정 및 최적화

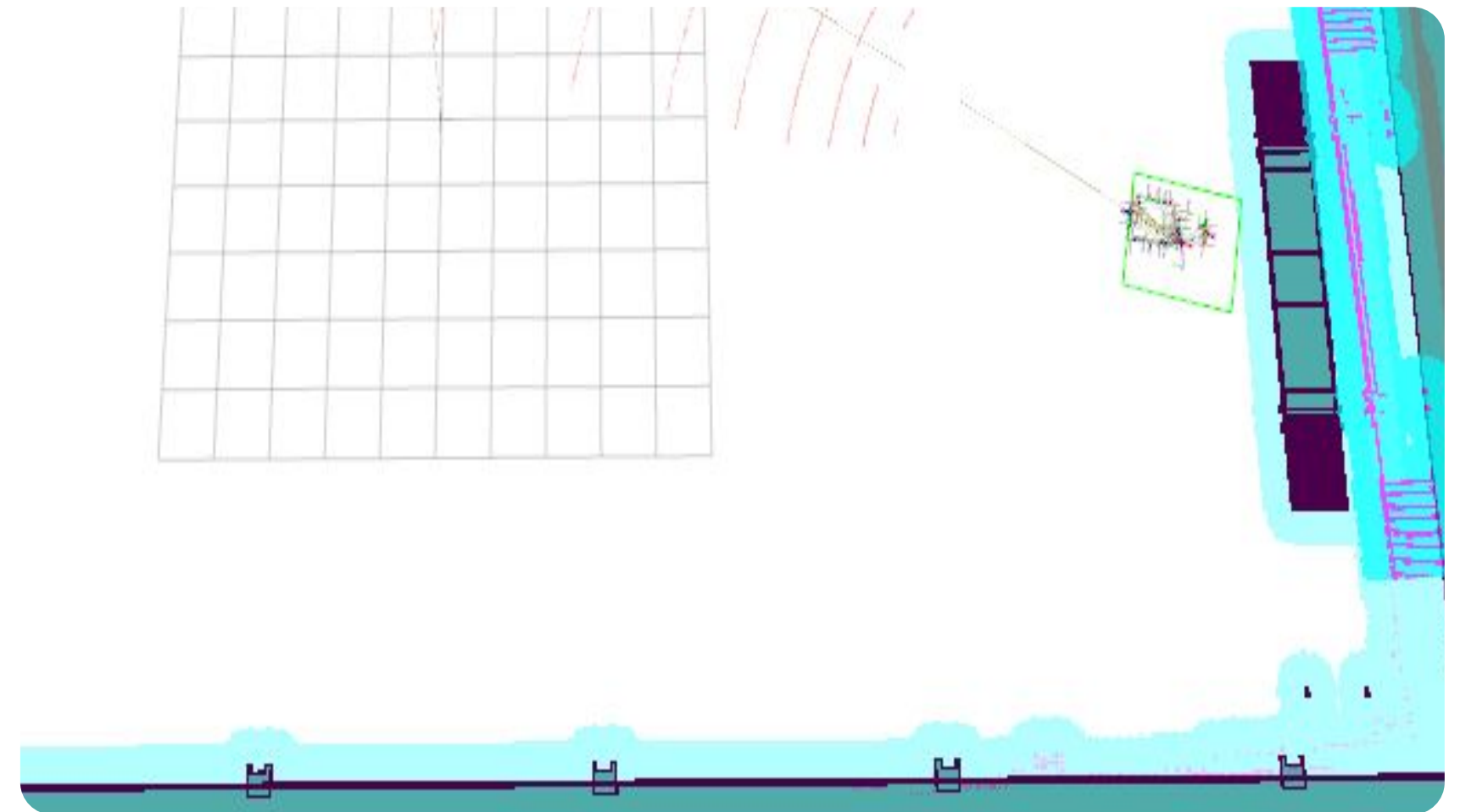
1. xy_goal_tolerance: 0.25 → 0.05

- 5cm 이내의 오차가 될 때까지 이동

2. initial_pose y: -1.0 → -2.0

- 초기 Y 좌표 보정

→ 라이다 센서와 정확하게 일치



맵 좌표 그리드 정합 후

| PC 2: 모듈 - move

STEP 01

imu 센서를 통해
넘어진 걸 파악

STEP 02

강화학습된 pt 파일로
로봇팔을 조종

STEP 03

imu 센서가
일어섬을 감지

STEP 04

잠시 정지하여 라이다 센서로
현재 위치 파악

STEP 05

이후 목적지로 재 이동

자세 복구 — 강화 학습 이용 프로세스

| PC 2: 모듈 - move

자세 복구 — 강화 학습 이용

[시도 내용]

roll, pitch = $\pm 1.57 \sim 3.14$ 로 설정하여 뒤집어진 상태로 소환, 로봇팔을 이용하여 자세 복원 강화학습 시도

[한계 및 분석]

- 강화학습 환경에선 성공하였으나 시나리오 환경에선 미세한 차이로 일어나지 못함
- 학습 시에는 움직이는 상태에서 복원을 시작했으나, 시나리오상에선 정지 상태에서 시작하여 생긴 오차로 판단

→ 강화학습 환경 재설정 후 신규 학습 진행 중



| PC 2: 모듈 - move

자세 복구 — 강화 학습 이용

[분석 및 최적화]

시나리오 환경 충격 구현 시, 1회를 제외한 모든 상황에서 좌우 90도 전도 확인

[개선 결과]

- 재학습 시간 절약을 위해 스폰 상태를 ± 1.57 로 제한
- 1/6 iteration으로 기존과 동일한 동작 구현 성공

→ 넘어진 상태에서 기립 동작 완벽 구현 성공



SECTION 4.4

통신

인식과 추론, 제어를 연결하는 **Semantic AI** 기술 구조 소개

| 3PC 통신



소켓(Socket) 및 ROS2 FastDDS 기반의 초고속 데이터 연동 아키텍처

- **PC1 작업지시 LLM:** 신속한 지시문 생성 및 소켓 포트 9999를 통한 비동기 지시 송신
- **PC2 코더 LLM:** 처리 완료된 피드백 및 상태 데이터를 포트 8889를 통해 PC 1로 실시간 전송
- **PC3 아이작심:** 생성된 코드와 제어 명령을 가상 3D 로봇 환경(Isaac Sim)에서 물리 검증 및 작업 수행

| 3PC 통신을 분산한 이유



하드웨어 자원(VRAM) 고갈 방지 및 독립성 확보

PC1 (DeepSeek-R1 14B): 고도의 추론 및 지시 생성을 위해 대용량 VRAM 독점 사용 필요.
PC2 (Qwen-Coder 30B + Florence 2): 코드 생성과 비전 인지 모델이 동시에 상주하므로 단독으로도 이미 RTX 5080의 가용 자원을 한계까지 점유.
PC3 (Isaac Sim): 실시간 3D 가상 환경 렌더링 및 물리 연산은 인색적인 GPU 부하 발생.

상황별 데이터 특성에 맞춘 통신 프로토콜 이원화

AI 인터페이스 구간 (PC1 ↔ PC2): 데이터 유실이 없어야 하는 고용량 텍스트 중심의 통신이므로 명확한 TCP 소켓 통신 채택.
로봇 시뮬레이션 구간 (PC2 ↔ PC3): 초고속 실시간 제어 명령 핵심이므로 ROS2 FastDDS 채택.

**결론: 단일 PC에 통합 구동 시 메모리 초과(OOM)와 전체 시스템 다운이 불가피하므로,
물리적 연산 노드 분리는 필수적 선택이었습니다.**

SECTION 4.5

화성 맵 제작

가상 환경 맵 제작

| 화성 가상 환경 제작 개요



실감형 가상 우주 탐사 환경 제작

화성의 지형적 특성과 물리적 환경을 정밀하게 묘사함으로써, VLM(Florence AI)의 시각적 인지 회피 알고리즘 검증을 위한 고정밀 베이스 데이터 기반의 환경 구축

▶ 비전 인지 최적화 고해상도 맵

Florence AI의 지형 판독 및 장애물 회피 알고리즘 검증을 위한 고정밀 메시 데이터 기반의 환경 구축

▶ 광원 정합성을 통한 인지 오류 방지

자체 제작한 HDR 스카이돔과 OmniPBR Material을 적용하여, 통상 특수 거친 음영으로 발생할 수 있는 AI 인지 성능 저하 차단

▶ 동역학 기반 센서 데이터 고증

물리 엔진 내 화성 중력 설정을 통해 로봇 주행 시 발생하는 카메라 피치·롤(흔들림)을 물리적으로 재현하여 AI 입력 데이터의 현실성 확보

I 화성 가상 환경 제작: 고해상도 지형 매쉬

방식 1: 절차적 알고리즘

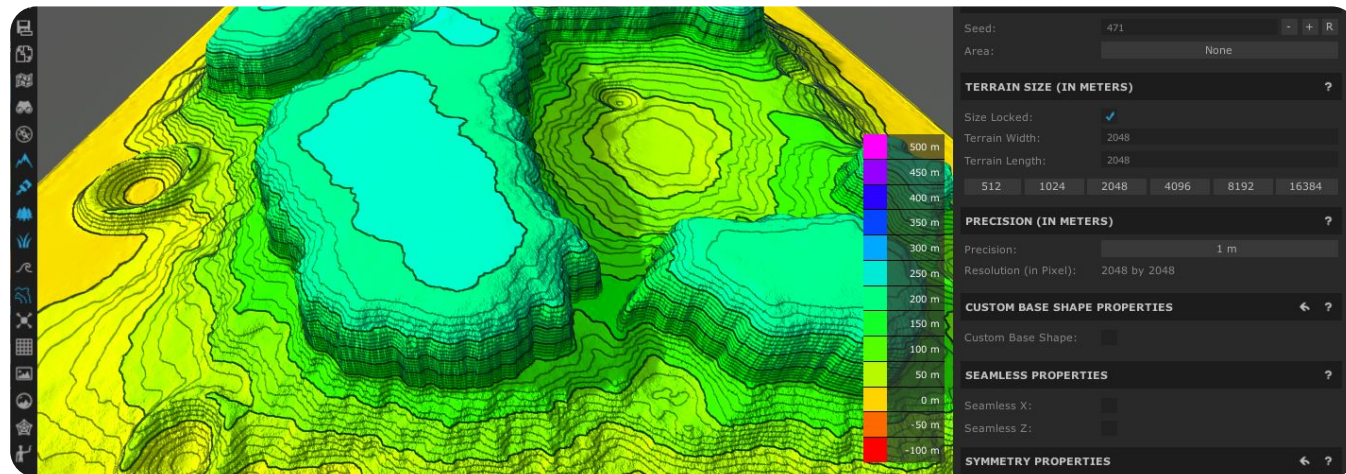
World Creator 기반 생성

프로세스: World Creator → 메쉬 추출 → Isaac Sim 이식

특징: 절차적 생성 모델 및 노이즈 기반 광대역 지형 구현

Florence 인지: 대규모 모래도 노이즈로 매력 생성된 정밀한 지형 굴곡이 메쉬 표면에 그대로 살아있어, 광대역 지형 전체에 균일하고 리얼한 폴리 접촉면 제공

Isaac Sim 효과: 수학적 노이즈에 의해 생성된 정밀한 지형 굴곡이 메쉬 표면에 그대로 살아있어, 광대역 지형 전체에 균일하고 리얼한 물리 접촉면 제공.



방식 2: 시나리오 맞춤형

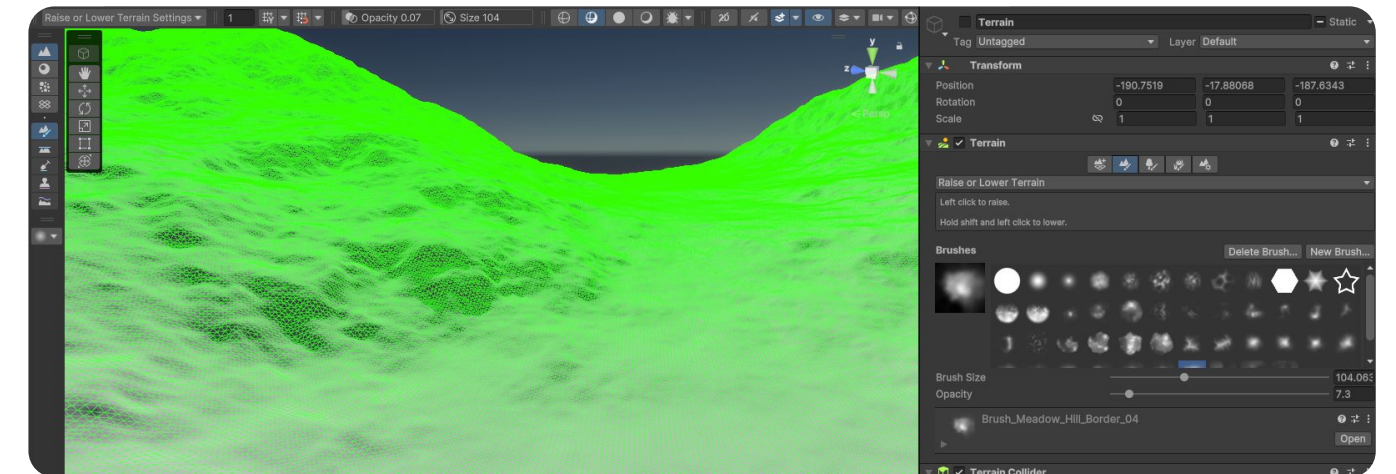
Unity Terrain → Terrain to Mesh (유료 에셋)

프로세스: 유니티 테레인(브러시 스컬프팅) → 메쉬 추출 → Isaac Sim 이식

특징: 장애물 링크, 경사각, 협곡 폭 등 의도에 맞춘 정밀 설계

Florence 인지: 균형 메쉬 추출 후, 엔지니어가 주행 시나리오에 맞춰 충돌 검증 필수 구역만 정렬한 폴리곤을 밀집시키고 불필요한 외각은 깎아내는 등, 연산 부하를 효율적으로 제공

Isaac Sim 효과: 엔지니어가 주행 시나리오에 맞춰 중요 구역만 폴리곤을 밀집시키고 불필요한 외각은 깎아내는 등, 연산 부하를 효율적으로 제공.



I 화성 가상 환경 제작: PBR 텍스처 및 중력

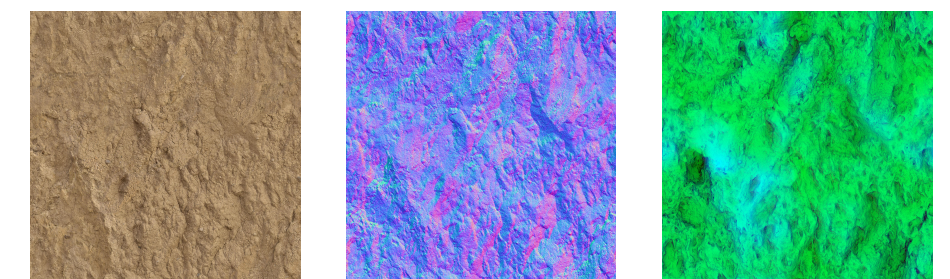


가상 우주 탐사를 위한 화성 환경 구현

HDR을 생성하여 DomeLight로 화성에 온 듯한 환경을 구성. Albedo, Normal, Ambient Occlusion 맵 텍스처로 OmniPBR Material 구성



생성형 AI 활용 HDR 이미지 생성



Albedo

Normal

AO

화성용 PBR 4K 텍스처(외부 에셋)

아이작심 PhysicsScene 프로퍼티의 Gravity Magnitude를 화성 중력에 맞게 설정

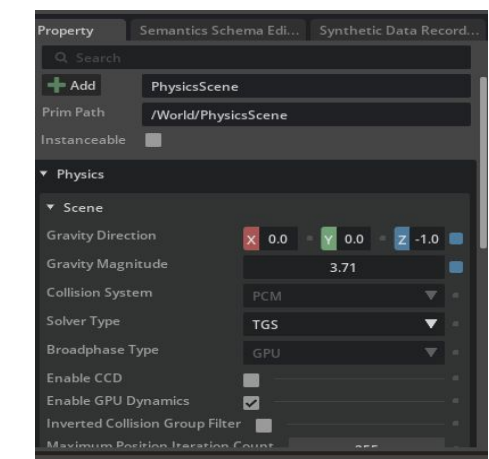
Gravity Magnitude를 Earth Gravity에서 3.71 혹은 3.72로 변경

$$g = G \cdot M / R^2$$

G: 만유인력 상수
 $g_{mars} = 6.6741 \times 10^{-11} \times 6.4171 \times 10^{23} / (3.3895 \times 10^6)^2 \approx 3.729 \text{ m/s}^2$
M: 행성 표면 중력 가속도 / R: 행성의 반지름



아이작심 화성 환경 및 중력설정



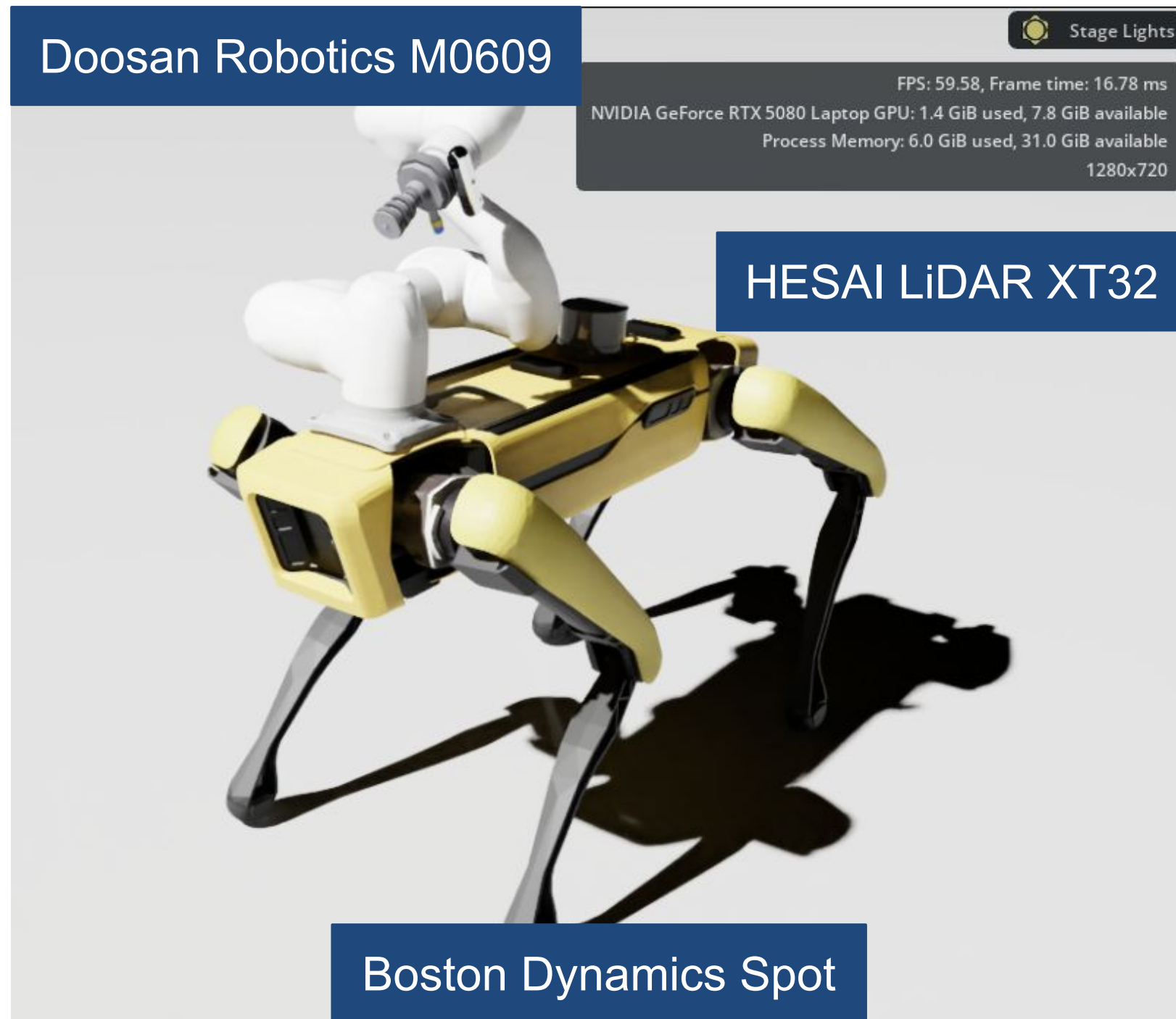
실제 화성은 완벽한 구형이 아니라 적도 부분이 약간 부풀 타원체이며, 실험 환경 그래픽 부하 정도를 고려하여 3.71의 시뮬레이터로 통일.

SECTION 4.6

프로젝트 미구현 기능

프로젝트를 고도화 하기 위해 시도한 흔적

| AMR 모델 변경 Boston Dynamics Spot



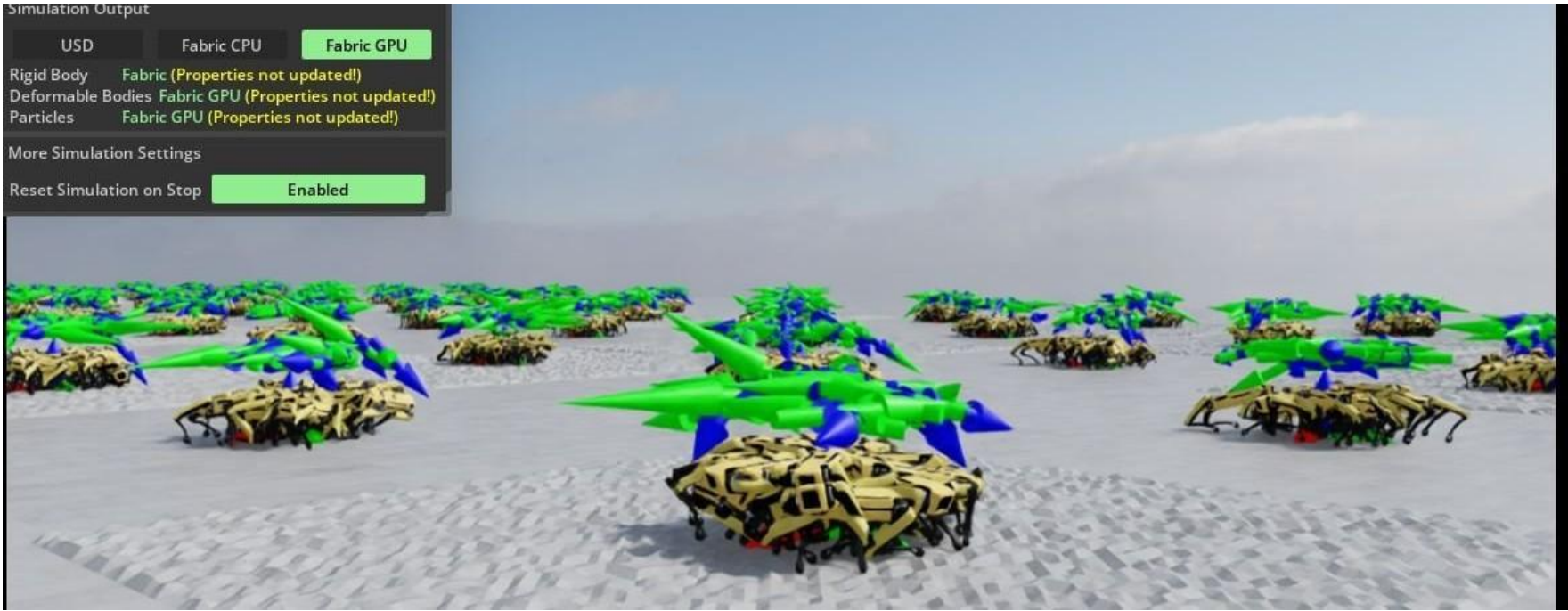
Nova Carter와 비교

- 굴곡이 있는 지면에서 유리
- Task 수행 중 고정 안정성 확보

수정 사항

- Boston Dynamics 최대 적재하중 **14kg**에 맞춰 Doosan Robotics 무게 **27kg → 5kg** 감소
- Walk 강화학습시 M0609 Joint 강체 설정

I Spot Walking 강화학습

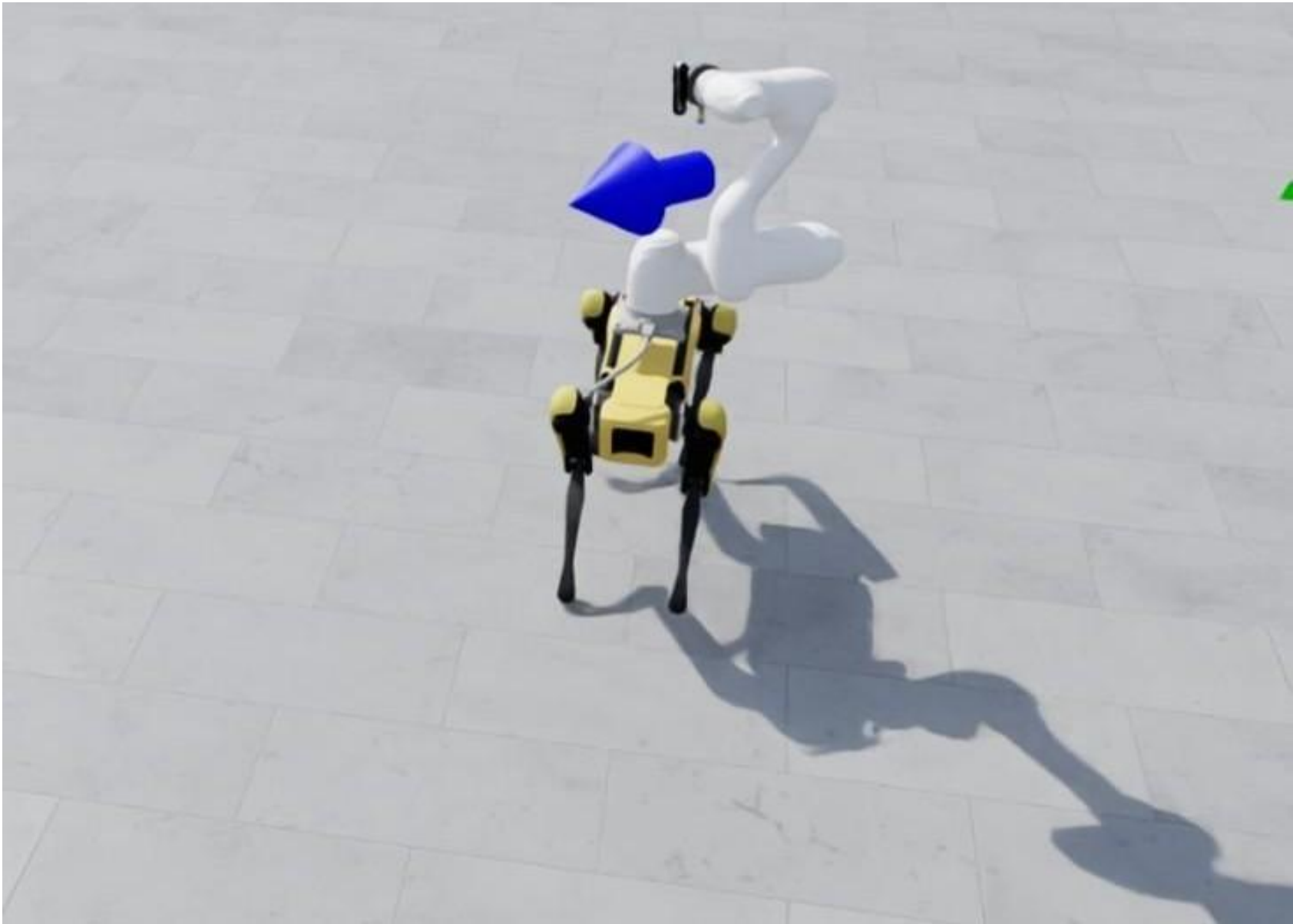


상점 (Reward)	
걸음걸이 패턴 일치율	+ 10.0
명령 선형 속도 추종	+ 5.0
명령 회전 속도 추종	+ 5.0
다리 체공 보상	+ 5.0
발 높이 제어	+ 0.5

벌점 (Penalty)

정지, 기본자세 이탈	- 0.7
몸체 기울어짐, 흔들림	- 3.0
모터 명령의 부드러움	- 2.0
다리 체공 균형 분산	- 1.0
몸체 불필요한 급가속	- 1.0
발 미끄러짐	- 0.5
관절 과속도	- 0.01
관절 과가속도	- 0.0001

I Spot Walking 1차 학습 결과 및 학습내용 수정



수정 내용

- 정지 명령 벌점 강화
- 몸체 기울어짐 패널티 완화
- 몸체 순간 흔들림 패널티 완화

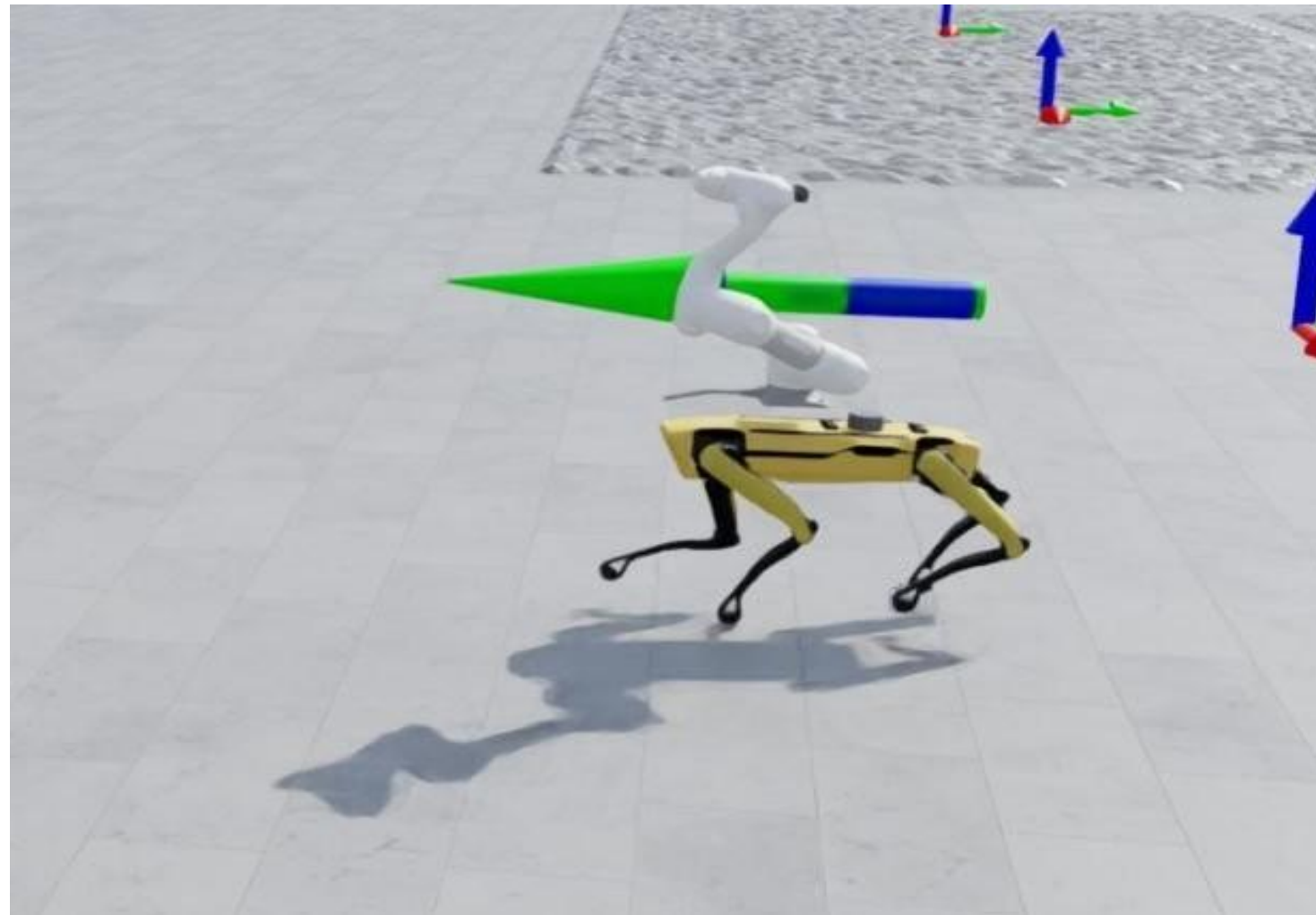
벌점 변경

정지, 기본자세 이탈	- 0.7 ⇒ - 1.5
몸체 기울어짐, 흔들림	- 3.0 ⇒ - 1.5
모터 명령의 부드러움	- 2.0 ⇒ - 1.0
다리 체공 균형 분산	- 1.0
몸체 불필요한 급가속	- 1.0
발 미끄러짐	- 0.5
관절 과속도	- 0.01
관절 과가속도	- 0.0001

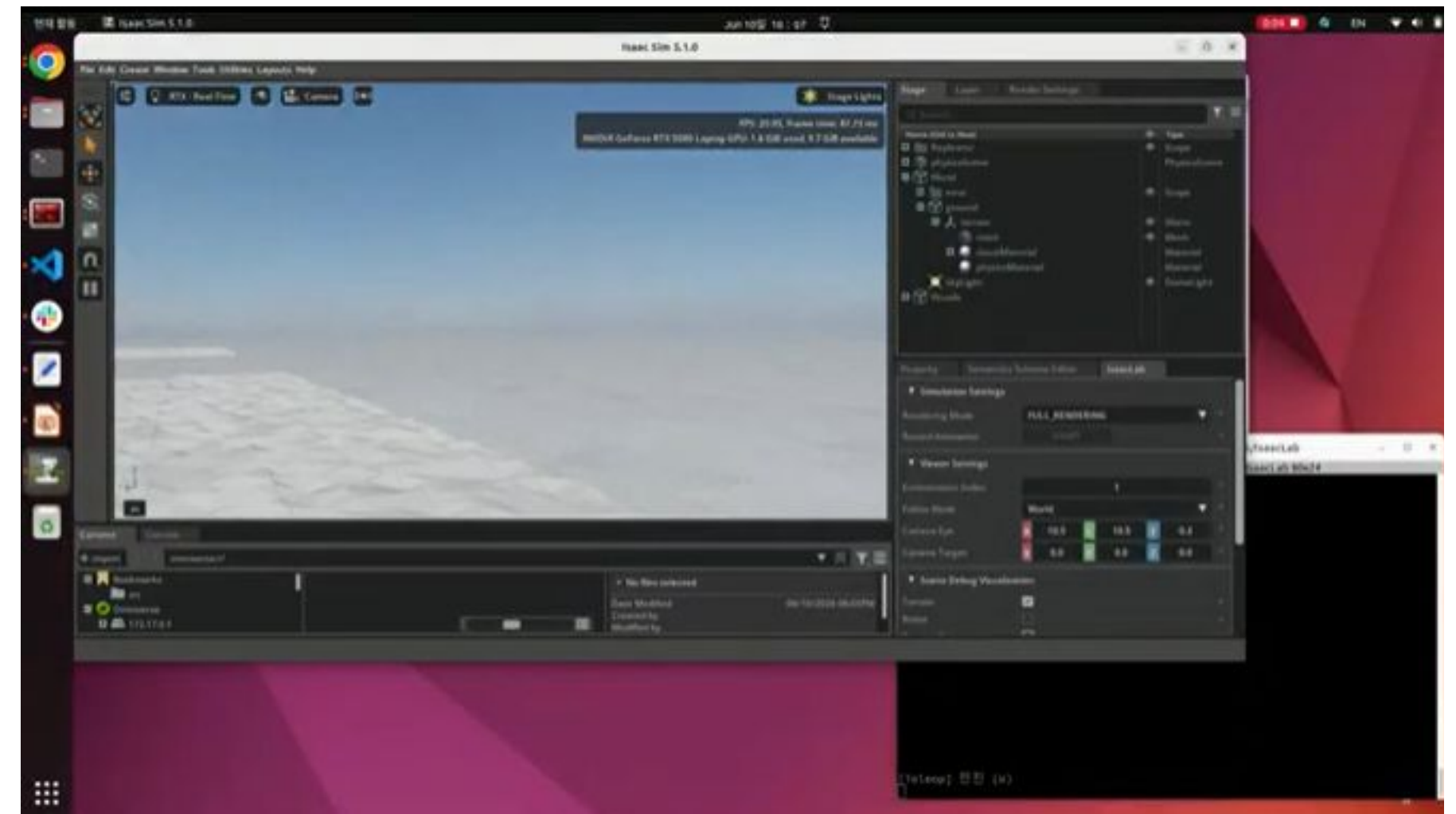
⚠ 문제점

- 로봇 모델 구조 붕괴
- idle 상태 무게중심 불균형으로 인한 전방 우측 쏠림

I Spot Walking teleop 기능 추가 및 학습 결과



[teleop] 키보드 조작 안내
W / S : 전진 / 후진
A / D : 좌회전 / 우회전
↑ / ↓ 방향키 : 속도 증가 / 감소
현재 속도 : 0.5 m/s (범위: 0.1~5.0)



CHAPTER 5

자체 평가 의견

춘식이 화성가즈아 프로젝트 자체 평가

| 프로젝트 성과 및 기술적 한계

주요 성과 (Achievements)

Semantic AI 기반 자율 작업 & 멀티 AI 통합

자율 작업 구조 구현 및 멀티 AI 통합 시스템 구축, 자연어 기반 작업 수행 및 **Local AI 환경** 내 완전 구현

분산 시스템 안정성 확보

3-PC 분산 처리 환경에서 Hermes Self-Correction, Memory Log, Guardrail 시스템 등을 통해 안정성을 확보함

강화학습 / Pick & Place 실험 완성

가상 환경에서 Pick & Place 작업의 강화학습 시도 및 Spot Walking 학습 환경 구축

기술적 한계 (Limitations)

플랫폼 의존성 및 진입 장벽

Linux OS, 그래픽 드라이버 커널, Isaac Sim 버전 별 API 호환성 등에 매우 엄격하여 초기 인프라 구축 및 의존성 관리(Dependency Hell) 측면에서 높은 오버헤드 발생

실시간 동역학 연산의 리소스 병목

대규모 VRAM 요구량 및 PhysX 엔진의 실시간 병목으로 인해 시스템 정지, 메모리 누수(OOM), 프로세스 비정상 종료 현상이 빈번하여 하드웨어 가용 자원의 극한 단계까지 제어가 필요했음

비전 데이터 및 안전/예외 처리 한계

학습된 객체 외 조작 불가 및 복잡한 상황 대응 미비, Safety 시스템 추가 검증 필요, 동작 시 예외 처리 부족 및 조명 변화 시 시스템 의존성 상존

| 프로젝트 자체평가 및 개선 방향

★★★★☆ 4.0 / 5.0

👍 잘된 점

- 디지털 트윈 기반 실험 환경 구축
- 3-PC 분산형 VLA 구조 구현
- AI 기반 Task Packet 생성
- 강화학습 / Pick & Place 실험 시도

⚠️ 아쉬운 점

- Isaac Sim 개발 환경 불안정
- 로봇 동작 안정성 한계
- 3D 환경 구성 및 객체 제어 어려움
- 학습 시간 / 하드웨어 성능 제약

📈 개선 방향

- 고성능 GPU 환경에서 재학습
- 다양한 초기 조건 기반 학습
- PC2 실행 피드백 연동
- 실제 로봇 / 고정밀 디지털 트윈 확장

—

Q & A

감사합니다.



Adios!

Page 10 of 10

감사합니다.

Github : [AssetHelper/rokey7_c3_lets_go_to_mars](https://github.com/AssetHelper/rokey7_c3_lets_go_to_mars)

YouTube : [두산로보틱스 부트캠프 C3 춘식이화성가즈아](#)